

A Byzantine-robust asynchronous federated learning method on Non-IID data

Chen Fang^a, Fang Yang^a, Sensen Li^a, Kunyuan Fan^a, Yuanbo Guo^b, Yanmi Wu^{a,*} 

^a Information Engineering University, Zhengzhou, 450001, Henan, China

^b School of cryptography, Hainan University, Haikou, 570228, Hainan, China

HIGHLIGHTS

- Proposes BRAFed, a Byzantine-robust asynchronous FL framework for Non-IID data, poisoning attacks, and delays.
- Designs an asynchronous-aware detector using adaptive Mahalanobis distance and projection-based delayed-gradient correction.
- Provides convergence analysis of delayed gradients, malicious updates, and data heterogeneity on model utility.
- Shows superior robustness and accuracy on CIFAR-100, Shakespeare, and COVIDx under poisoning, Non-IID data, and delays.

ARTICLE INFO

Communicated by G. Yu

Keywords:

Asynchronous
Poisoning attack
Non-IID
Federated learning

ABSTRACT

Asynchronous federated learning (FL) improves training efficiency under heterogeneous devices, but it is more vulnerable to Non-IID data, poisoning attacks, and stale updates. To address these coupled challenges, we propose a Byzantine-robust asynchronous FL method on Non-IID data, termed BRAFed. On the client side, BRAFed introduces a model difference regularization to alleviate model drift caused by Non-IID data. On the server side, it develops an asynchronous-aware malicious gradient detection mechanism based on adaptive Mahalanobis distance, where a dynamic threshold combines round-dependent decay with the current gradient distribution. BRAFed further incorporates a projection-based correction strategy to align conflicting delayed gradients with the current optimization direction, thereby mitigating both poisoning attacks and straggler-induced degradation. We also provide a formal convergence analysis to characterize the effects of delayed gradients, malicious updates, and data heterogeneity. Extensive experiments on CIFAR-100, Shakespeare, and COVIDx under six poisoning attacks demonstrate that BRAFed consistently outperforms state-of-the-art defenses in both robustness and accuracy, especially in highly heterogeneous and delayed settings.

1. Introduction

Federated learning (FL), as an emerging distributed machine learning framework, enables clients (e.g., smartphones, personal computers, and IoT devices) to preserve privacy by exchanging model parameters rather than uploading raw local data to a central server. Most existing FL methods are synchronous [1–3], where the server waits for all selected clients to finish local training before updating the global model. However, in open IoT environments, heterogeneous computational capabilities and unstable network conditions often cause some clients to upload updates later than others, resulting in the *straggler problem*. Directly discarding

delayed updates can improve efficiency, but wastes client computation and may degrade model accuracy [4].

To alleviate this issue, asynchronous FL has been widely studied [5–7]. Unlike synchronous FL, the server updates the global model immediately after receiving one or a few client updates, thereby reducing idle waiting time and improving training efficiency. Representative studies include asynchronous federated optimization with staleness-aware averaging [5], asynchronous-adaptive local iteration control under time budgets [6], and online client selection for latency minimization with fairness considerations [7]. Despite these advantages, asynchronous FL still faces several critical challenges in practice.

* Corresponding author.

Email address: wym3529@126.com (Y. Wu).

<https://doi.org/10.1016/j.neucom.2026.134301>

Received 17 February 2026; Received in revised form 29 May 2026; Accepted 16 June 2026

Available online 17 June 2026

0925-2312/© 2026 Elsevier B.V. All rights reserved, including those for text and data mining, AI training, and similar technologies.

First, delayed updates may conflict with current updates. In asynchronous environments, the server may receive gradients from the current round together with stale gradients from previous rounds. Existing methods mainly alleviate this issue through staleness-aware weighting [8–10]. However, when staleness becomes large, delayed updates may differ substantially from current ones in both direction and magnitude, and simple weight adjustment is often insufficient to eliminate the resulting optimization bias. Therefore, handling the directional conflict of delayed updates is crucial for improving asynchronous FL accuracy.

Second, Non-IID data further aggravates the difficulty of model convergence. In practical FL systems, local client data are often highly heterogeneous, causing significant divergence among client updates and making global aggregation unstable. Some studies mitigate Non-IID effects via clustering [11] or knowledge distillation [12], but these methods are not readily applicable to asynchronous FL. Other works have attempted to jointly address staleness and data heterogeneity [13,14]; nevertheless, they still do not explicitly resolve directional conflicts introduced by delayed updates.

Third, defending against poisoning attacks in asynchronous FL remains challenging. Byzantine clients may poison local data or upload malicious gradients to mislead global optimization. Although robust FL methods have been proposed for poisoning defense [15,16], they generally struggle in asynchronous settings because delayed benign updates and malicious updates may both deviate substantially from the current model. While several studies have explored Byzantine-robust asynchronous FL [17–21], they typically assume IID data. Under Non-IID data, the combination of client heterogeneity and asynchronous delay enlarges the deviation of benign updates, making it even harder to distinguish them from malicious ones.

To address the above challenges, we propose a Byzantine-robust asynchronous FL method on Non-IID data, termed BRAFed. The core idea is to jointly handle data heterogeneity, gradient staleness, and malicious updates within a unified design. On the client side, a model difference regularization is introduced to mitigate model drift caused by Non-IID data. On the server side, BRAFed constructs a reliable reference from current gradients for adaptive malicious gradient detection, and further incorporates conflict-aware handling of delayed updates to ensure consistency with the current optimization direction. To the best of our knowledge, this is among the first efforts to simultaneously address the straggler problem, data heterogeneity, and poisoning attack defense in asynchronous federated learning. The main contributions of this work are summarized as follows:

1. We introduce an asynchronous-aware malicious gradient detection mechanism based on an adaptive Mahalanobis distance, where the distribution of benign gradients is dynamically estimated and a time-varying threshold accounts for evolving Non-IID statistics. A subsequent projection-based correction aligns delayed gradients with current updates, resolving directional conflicts and improving aggregation consistency, thereby mitigating both poisoning attacks and straggler-induced performance degradation.
2. We provide a formal convergence analysis for BRAFed in the non-convex setting, explicitly characterizing the impact of delayed gradients, malicious updates, and Non-IID data. The analysis reveals that these factors interact in a non-trivial manner, where data heterogeneity amplifies the effect of staleness and imperfect detection introduces a non-vanishing error floor, offering theoretical insights into robust asynchronous FL design.
3. We conduct comprehensive experiments on multiple datasets under various poisoning attacks, Non-IID levels, and delay ratios. The results demonstrate that BRAFed consistently outperforms state-of-the-art methods in terms of robustness and accuracy, particularly in challenging scenarios where data heterogeneity, client delay, and adversarial behavior coexist.

2. Related work

2.1. IID data-based byzantine-robust FL

To defend against poisoning attacks, many Byzantine-robust FL methods have been proposed by discarding or downweighting malicious gradients during global aggregation. Existing methods can be broadly divided into two categories: those based on statistical distance [22–25] and those based on performance detection [26–29]. Representative works include Krum [22], which removes updates far from the majority using Euclidean distance, and Trimmed Mean [27], which filters extreme values before averaging. Shen et al. [24] combined median statistics with Pearson correlation to assign robust aggregation weights, while Sun et al. [25] jointly exploited model-domain and data-domain distances to separate Byzantine and benign clients.

Another line of work relies on additional supervision or feature-based detection. Kasyap et al. [26] used a clean server-side dataset to build a trusted base update and assign trust scores via cosine similarity, while Gong et al. [27] also assumed access to clean server data and employed explainable AI techniques to enhance malicious-update suppression. Shejwalkar et al. [28] combined random sampling and spectral filtering for Byzantine removal, but required the number of malicious clients to be known in advance. Jiang et al. [29] assessed client data quality through latent feature distributions extracted by a lightweight generator. However, these methods are designed for IID settings. Under Non-IID data, benign gradients may naturally deviate from one another in a way similar to malicious updates, which often leads to false detection.

2.2. Non-IID data-based byzantine-robust FL

To address Byzantine robustness under Non-IID data, Han et al. [30] observed that benign models tend to share similar top- k and bottom- k important parameters even under heterogeneous distributions, and designed a weighted robust aggregation strategy based on this property. Zuo et al. [31] leveraged the geometric median to aggregate local gradients in federated learning, providing robustness against Byzantine clients under Non-IID data. Chen et al. [32] measured similarity among client updates and assigned very small weights to anomalous models, while further purifying the aggregated model with local clean data. Qiao et al. [33] proposed FatCC, which integrates local logit calibration and global feature contrast into federated adversarial training to improve both clean and robust accuracy under Non-IID data. While FatCC demonstrates effectiveness in enhancing general robustness against adversarial attacks, it does not address the poisoning attacks considered in this work. Zhao et al. [34] utilized a public shadow dataset to construct a server-side supervision anchor, then combined secure aggregation with dynamic dual filtering based on cosine similarity and L2 norm to eliminate malicious updates on Non-IID data.

However, most of these methods are designed for synchronous FL, where the server waits for all selected clients before aggregation, making training vulnerable to stragglers. In asynchronous FL, current and delayed updates may coexist, and the larger divergence between them makes it much harder for existing Byzantine defenses [30–32,34] to distinguish stale benign updates from malicious ones.

2.3. Asynchronous byzantine-robust FL

To defend against poisoning attacks in asynchronous FL, Yang et al. [17] adopted a buffered aggregation strategy, performing robust aggregation only after buffers were filled. Fang et al. [18] compared each incoming update with a server-computed trusted reference to distinguish poisoning attacks from delayed benign updates. Bai et al. [19] dynamically adjusted aggregation weights based on the similarity between local and global updates, and further introduced bidirectional credibility cross-checking among clients, but still relied on a trusted validation set in the initial stage. To remove this dependency, Joo et al.

Table 1
The works comparison.

Methods	Support data heterogeneity	Asynchronous FL	Poisoning attack defense	Require no trusted dataset
[18]	×	✓	✓	×
[19]	×	✓	✓	×
[20]	×	✓	✓	✓
[21]	×	✓	✓	✓
[25]	×	×	✓	✓
[27]	×	×	✓	×
[29]	×	×	✓	✓
[31]	✓	×	✓	✓
[32]	✓	×	✓	✓
[34]	✓	×	✓	✓
Ours	✓	✓	✓	✓

[20] used layer-wise correlation and staleness-aware weighting to filter malicious gradients without requiring trusted data during training. Ma et al. [21] combined asynchronous gradient updates with regularization and an L1 penalty to improve Byzantine robustness under staggered communication.

Nevertheless, all these asynchronous Byzantine-robust FL methods [17–21] are not designed for Non-IID settings. As a result, poisoning defense in asynchronous FL under heterogeneous data remains insufficiently explored. Finally, we summarize the above methods in Table 1.

3. Proposed method

3.1. System model

This paper considers constructing a Byzantine robust FL framework in heterogeneous IoT systems, where clients' data exhibit Non-IID characteristics. Meanwhile, clients also differ in terms of computational capabilities and communication bandwidth, while confronting the challenge of poisoning attacks. As shown in Fig. 1, the system model of BRAFed consists of a central server, online clients and delayed clients. The role of each entity is described as follows:

1. Central server: In the initialization phase, the server distributes the initial model parameters to all clients. In each training round, the server collects gradients from both online and delayed clients, corrects the delayed gradients, and detects any malicious gradients among them. Finally, the server aggregates the remaining benign gradients and distributes the aggregated results to all clients.
2. Online clients: In each round, online clients receive the latest global model from the server, conduct local training based on their local datasets, and then upload their local gradients to the server.
3. Delayed clients: Due to limitations in local computational capacity or communication bandwidth, the local gradients of delayed clients are not received by the server in the current training round, but instead arrive after multiple rounds have elapsed.

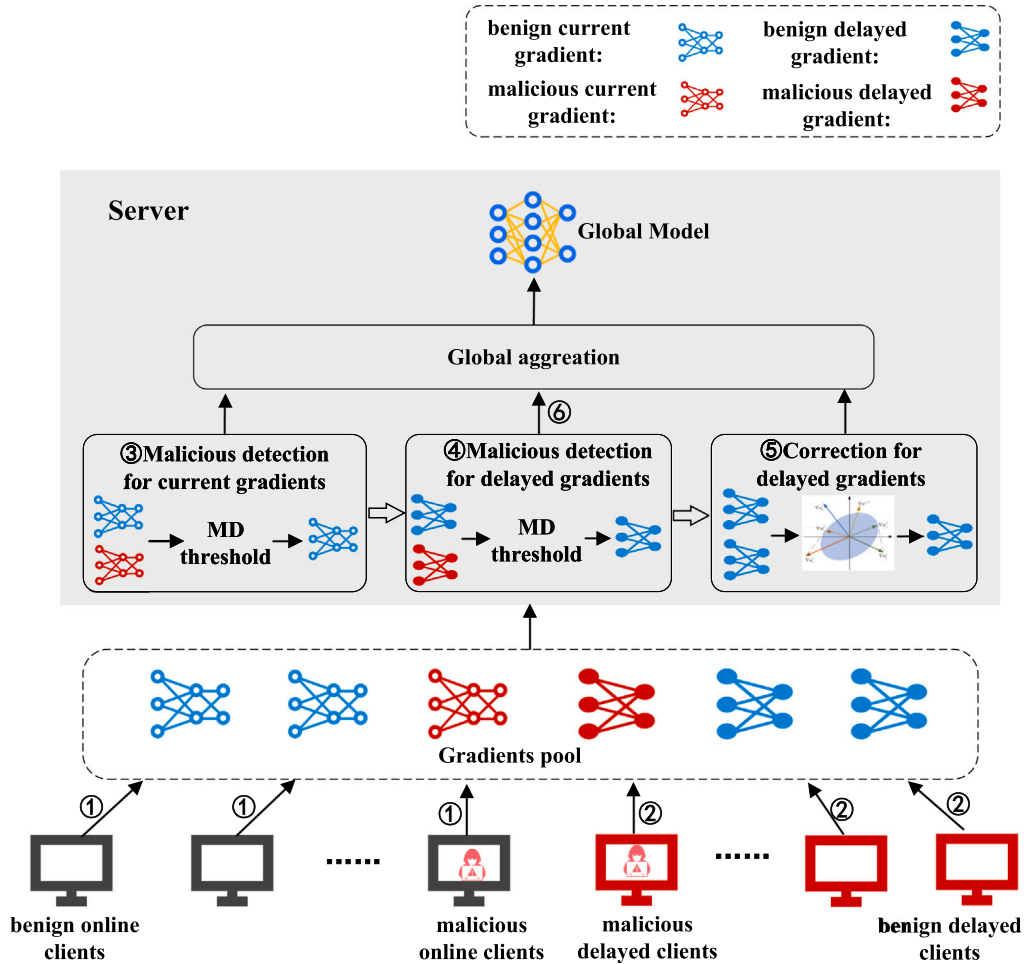


Fig. 1. The system model of our scheme.

Specifically, in training round τ , the online client C_i obtains the gradient ∇w_i^τ via local training and sends ∇w_i^τ to the server (Step ①). Besides receiving gradients from online clients, the server may also receive delayed gradients $\nabla w_j^{\tau_j}$ from previous training round $\tau_j (\tau_j < \tau)$ (Step ②). For the current gradients $\nabla W_o^\tau = \{\nabla w_1^\tau, \nabla w_2^\tau, \dots, \nabla w_p^\tau\}$, the server detects potential malicious gradients therein (Step ③). For delayed gradients $\nabla W_d^\tau = (\nabla w_0^\tau, \nabla w_1^{\tau_1}, \dots, \nabla w_d^{\tau_d})$, the server first detects potential malicious gradients therein (Step ④), and then corrects the update direction of benign delayed gradients (Step ⑤). Finally, the server conducts weighted aggregation on the remaining benign current gradients and benign delayed gradients to derive the global model (Step ⑥).

3.2. Threat model

In our system, we assume that the server is honest-but-curious, meaning the server may infer the clients' private information through their local models. Clients are categorized into online clients and delayed clients, both of which may include Byzantine clients compromised by adversaries, potentially contaminating local data or uploading malicious model updates. The potential threats arising from these entities are as follows:

1. **Difficulty in model convergence:** Since clients' data follows a Non-IID distribution, local models may drift, which in turn makes the aggregated global model difficult to converge.
2. **Decreased accuracy:** Byzantine clients may launch data poisoning attacks or model poisoning attacks against asynchronous FL. Coupled with the potential directional conflicts between delayed gradients and current gradients, these issues can significantly reduce the accuracy of the global model.

3.3. Problem definition

Consider a heterogeneous IoT system with n clients $C = \{C_1, C_2, \dots, C_n\}$ participating in FL, each possessing a local dataset denoted as D_i . The goal of FL is to train a global model $w \in \mathbb{R}^d$ by $\min_{w \in \mathbb{R}^d} F(w)$, where $F(w) = \frac{1}{n} \sum_{i \in [n]} f(w; d_i)$, d_i is sampled from the local dataset D_i of client C_i . These clients exhibit both data heterogeneity and device heterogeneity. In terms of data heterogeneity, the data from different clients present Non-IID features. For instance, in client C_i 's dataset, samples with label l_i account for 80%, while those with label l_j constitute 20%; whereas in client C_j 's dataset, samples with label l_i account for 20%, and those with label l_j account for 80%. Such Non-IID characteristics will lead to the local gradients trained by each client exhibiting significant differences. When these gradients are aggregated by the server, the global model may struggle to converge.

In terms of device heterogeneity, clients may have different computational capabilities or communication bandwidth, leading to variations in the time taken to upload their local gradients to the server. We denote the delayed clients in round τ as C_d^τ , and the online clients as C_o^τ . In the synchronous aggregation phase of the original FL algorithm, the server must wait for all delayed clients, resulting in low training efficiency. One alternative approach is for the server to aggregate only the gradients from online clients and discard those from delayed clients. However, this would result in the loss of valuable information that could be contained in delayed gradients. A more reasonable approach is to adopt a timestamp-based asynchronous aggregation strategy [8–10]. Suppose that in training round τ , the server receives delayed gradients $\nabla W_d^{\tau_j}$ from delayed clients $C_d^{\tau_j}$ in round τ_j and gradients ∇W_o^τ from online clients in current round. We will assign a lower weight to the delayed gradients $\nabla W_d^{\tau_j}$ based on the number of delayed rounds $\tau - \tau_j$ for weighted aggregation. However, if $\tau - \tau_j$ is excessively large, the delayed gradients are considered too stale and are likely to deviate significantly from the update direction of the current gradients. In such cases, directly incorporating these delayed gradients into global

aggregation could impair model accuracy. To address this, we introduce a function $D(\nabla w_i^\tau, \nabla w_j^{\tau_j})$ to quantify the deviation between ∇W_o^τ and $\nabla W_d^{\tau_j}$:

$$D(\nabla w_i^\tau, \nabla w_j^{\tau_j}) = \nabla w_i^\tau \cdot \nabla w_j^{\tau_j} \quad (1)$$

where $\nabla w_i^\tau \in \nabla W_o^\tau, \nabla w_j^{\tau_j} \in \nabla W_d^{\tau_j}$. If $D(\nabla w_i^\tau, \nabla w_j^{\tau_j}) < 0$, $\nabla W_d^{\tau_j}$ is deemed to conflict with ∇W_o^τ . How to handle these conflicting gradients is a key issue that needs to be addressed for improving model accuracy.

In addition to asynchronous delay, we further consider two practical client-availability dynamics: client dropout and client join. Dropout refers to the case where a client has been selected by the server in a given training round but is permanently unable to return its local update due to disconnection, energy depletion, or network failure. Client join refers to the case where previously unavailable clients become online and are added to the candidate client pool during training. These joining clients may hold heterogeneous local data and may also be benign or Byzantine. Therefore, unlike standard random client sampling, dropout and join explicitly change the set of available and responsive clients over time.

Besides, heterogeneous IoT systems may be susceptible to poisoning attacks from Byzantine clients, where adversaries upload malicious gradients to the server in an attempt to undermine the model's accuracy. Such malicious gradients could be present in either current or delayed gradients.

To address this coupled challenge, BRAFed first reduces local model drift on the client side via model discrepancy regularization, which alleviates the deviation caused by Non-IID data. Based on a more stable gradient distribution, the server then performs adaptive malicious gradient detection to separate suspicious updates from naturally deviated benign ones. For delayed benign gradients, BRAFed further resolves their directional conflicts with current updates through projection-based correction, thereby jointly improving robustness against poisoning attacks and stability under heterogeneous data.

3.4. Details of the proposed method

Algorithm 1 illustrates the overall training process of the proposed method, which consists of five main phases: system initialization, local model update, malicious gradient detection for online clients, malicious gradient detection and correction for delayed clients, and global model update. The technical details of each phase will be elaborated in the following subsections.

3.4.1. System initialization

The server randomly initializes the global model, which includes configurations such as the model structure, learning rate, and number of training rounds, and then distributes the initial model parameters to all clients.

3.4.2. Local model update

In training round τ , the server sends the global model w^τ to all clients. Client C_i selects a small batch data sample d_i from its local dataset D_i , then computes the loss of current data samples and uses gradient descent algorithm to optimize the local model for E iterations. To prevent the local model w_i^τ from excessively diverging from the global model w^τ when trained on Non-IID data, we incorporate the discrepancy between the global model and the local model into the local loss function as a regularization term, as shown in Eq. (2), which is mathematically identical to the technique used in FedProx [35]:

$$g(w_i^\tau; d_i) = f(w_i^\tau; d_i) + \frac{\epsilon}{2} \|w_i^\tau - w^\tau\|^2 \quad (2)$$

where $f(w_i^\tau; d_i)$ is the local function (i.e., cross entropy loss function), and $\epsilon > 0$ is the regularization coefficient. This is a widely used technique to mitigate the effects of statistical heterogeneity in federated

Algorithm 1: BRAFed.

Input: local datasets $D = \{D_1, D_2, \dots, D_n\}$, global training rounds T , regularization coefficient ε , sensitivity coefficient k , decay factor λ , gradient weight $\alpha_0, \alpha_1, \alpha_2$

Output: global model w^T

```

1 //Phase 1: System Initialization
2 Server randomly initializes global model  $w^0$ .
3 Server distributes  $w^0$  to all available clients.
4 for round  $\tau \in [0, T - 1]$  do
5   //Phase 2: Local model update
6   Server randomly selects a subset of clients  $C^\tau$  to participant in training.
7   for each client  $i \in C^\tau$  in parallel do
8      $\nabla w_i^\tau \leftarrow \text{LocalUpdate}(w^\tau, D_i, \varepsilon)$ 
9   end
10  //Phase 3: Malicious gradient detection for online clients
11  Let  $\nabla W_o^\tau$  be the set of gradients from online clients.
12   $\nabla W_o^\tau \leftarrow \text{MGDO}(\nabla W_o^\tau, k, \lambda)$ 
13  //Phase 4: Malicious gradient detection and correction for delayed clients
14  Let  $\nabla W_d^\tau$  be the set of delayed gradients
15   $\nabla w_m^\tau, \nabla w_{m_1}^\tau, \nabla w_{m_2}^\tau \leftarrow \text{MGDC}(\nabla W_d^\tau, \nabla W_o^\tau, \mu_{\text{online}}, \Sigma_{\text{online}}, T_{\text{online}}, \alpha)$ 
16  //Note:  $\mu_{\text{online}}, \Sigma_{\text{online}}, T_{\text{online}}$  are computed from  $\nabla W_o^\tau$  in MGDO
17  //Phase 5: Global model update
18   $w^{\tau+1} \leftarrow \text{GlobalUpdate}(\nabla w_m^\tau, \nabla w_{m_1}^\tau, \nabla w_{m_2}^\tau)$ 
19 end
20 return  $w^T$ 

```

learning, helping to prevent the local model from diverging too much from the global model. Thus the global objective function is defined as $G(w, D) = \min_{w \in \mathcal{D}} \mathcal{L}(w, D)$. After finishing local training, the resulting gradient, denoted as ∇w_i^τ , is sent to the server, as shown in Algorithm 2.

3.4.3. Malicious gradient detection for online clients

To defend against poisoning attacks launched by malicious online clients, we propose a malicious gradient detection method based on dynamic threshold, as shown in Algorithm 3. The MD is a method used to measure the distance between a point and a data distribution. Unlike simpler distance metrics such as Euclidean distance or cosine similarity, its key advantage lies in accounting for the correlations and scales across different dimensions of the data [36]. This makes it particularly suitable for detecting outliers in high-dimensional and correlated

Algorithm 2: LocalUpdate.

Input: Global model w^τ in round τ , local dataset D_i , local epochs E , regularization coefficient ε

Output: Local gradient ∇w_i^τ

```

1 Client  $i$  initializes its local model:  $w_i^\tau \leftarrow w^\tau$ 
2 for local epoch  $j \in [1, E]$  do
3   Sample a mini-batch  $d_i$  from local gradient  $D_i$ .
4   Compute regularized local loss:
      $g(w_i^\tau; d_i) \leftarrow f(w_i^\tau; d_i) + \frac{\varepsilon}{2} \|w_i^\tau - w^\tau\|^2$ 
5   Compute gradient:  $g \leftarrow \nabla_{w_i^\tau} g(w_i^\tau; d_i)$ 
6   Update local model:  $w_i^\tau \leftarrow w_i^\tau - \eta \cdot g$ 
7 end
8 Compute the local gradient:  $\nabla w_i^\tau \leftarrow w^\tau - w_i^\tau$ 
9 return  $\nabla w_i^\tau$ 

```

Algorithm 3: Malicious Gradient Detection for Online Clients (MGDO).

Input: Current gradients from round τ : $\nabla W_o^\tau = \{\nabla w_1^\tau, \nabla w_2^\tau, \dots, \nabla w_p^\tau\}$, sensitivity coefficient k , decay factor λ

Output: Benign current gradients ∇W_o^τ

```

1 Compute the mean vector  $\mu_{\text{online}}$  and covariance matrix  $\Sigma_{\text{online}}$  of the current gradients  $\nabla W_o^\tau$ 
2 Initialize an empty list  $MD_{\text{online}}$  to store MD values.
3 for each gradient  $\nabla w_i^\tau$  in  $\nabla W_o^\tau$  do
4   Calculate the MD:
      $MD(\nabla w_i^\tau) = \sqrt{(\nabla w_i^\tau - \mu_{\text{online}})^T \Sigma_{\text{online}}^{-1} (\nabla w_i^\tau - \mu_{\text{online}})}$ 
5   Append  $MD(\nabla w_i^\tau)$  to  $MD_{\text{online}}$ 
6 end
7 Compute the mean  $\mu_{MD}$  and standard deviation  $\sigma_{MD}$  of the values in  $MD_{\text{online}}$ 
8 Calculate the dynamic threshold:  $T_{\text{online}} = \mu_{MD} + k \cdot \sigma_{MD} \cdot e^{-\lambda t}$ 
9 Initialize an empty set  $\nabla W_o^\tau$  for benign current gradients.
10 for each gradient  $\nabla w_i^\tau$  in  $\nabla W_o^\tau$  do
11   if  $MD(\nabla w_i^\tau) \leq T_{\text{online}}$  then
12     Add  $\nabla w_i^\tau$  to the benign set  $\nabla W_o^\tau$ 
13   end
14 end
15 return  $\nabla W_o^\tau$ 

```

gradient distributions. Thus, we employ MD to quantify the deviation of a given gradient from the expected distribution, thereby identifying potential malicious gradients. Specifically, we first compute the mean vector and covariance matrix of the current gradients $\nabla W_o^\tau = \{\nabla w_1^\tau, \nabla w_2^\tau, \dots, \nabla w_p^\tau\}$ in round τ , where the covariance matrix captures the variance and correlation structure among them. In practice, directly computing the full covariance matrix for high-dimensional gradients is computationally expensive and potentially numerically unstable, particularly for modern neural networks with millions of parameters. To address this, we adopt a diagonal approximation, where only the variance of each parameter across clients is computed and stored along the diagonal. This reduces the computational complexity from $O(d^2)$ to $O(d)$, with d being the total number of model parameters, while avoiding matrix inversion. Empirically, this approximation maintains the effectiveness of Mahalanobis Distance-based detection while ensuring efficient and stable computation.

Based on this, the MD is then calculated for each current gradient using Eq. (3).

$$MD(\nabla w_i^\tau) = \sqrt{(\nabla w_i^\tau - \mu_{\text{online}})^T \Sigma_{\text{online}}^{-1} (\nabla w_i^\tau - \mu_{\text{online}})} \quad (3)$$

where μ_{online} and Σ_{online} are the mean vector and covariance matrix of all current gradients ∇W_o^τ , respectively, and T denotes the transpose operation. It can be inferred that gradients with larger MD values indicate greater deviation from the majority of the current gradients.

However, how to set an appropriate MD threshold for identifying malicious gradients remains an open challenge. In the early stages of FL, since most client data is Non-IID, the gradients from different clients may vary significantly, resulting in generally high MD values. As the training progresses, the gradients of the clients gradually converge in the same direction, leading to a decrease in MD values. To account for this dynamic change, we design a dynamic malicious detection threshold, which is jointly determined by the round-dependent decay term and the current distribution statistics of MD values. Let the MD values of all current gradients be $MD_{\text{online}} = MD_1, MD_2, \dots, MD_p$. The mean and standard

deviation of MD_{online} are calculated as $\mu_{MD} = \frac{1}{p} \sum_{i=1}^p MD_i$ and $\sigma_{MD} = \sqrt{\frac{1}{p-1} \sum_{i=1}^p (MD_i - \mu_{MD})^2}$. Then the threshold can be determined as follows:

$$T_{online} = \mu_{MD} + k \cdot \sigma_{MD} \cdot e^{-\lambda t} \quad (4)$$

where k is a sensitivity coefficient that controls the strictness of malicious detection, and λ is a decay factor that governs the rate at which the threshold decreases. Therefore, even in later training stages, if benign gradients remain dispersed under highly Non-IID data, such variability can still be partially reflected by μ_{MD} and σ_{MD} , preventing the threshold from shrinking independently of the actual gradient distribution. As a result, the adaptive threshold is designed to balance two competing objectives: maintaining sufficient sensitivity to malicious updates while reducing the risk of misclassifying naturally deviated benign gradients as malicious ones in later training stages.

By discarding current gradients with MD values greater than threshold T_{online} , the remaining current gradients are regarded as benign and denoted as $\nabla W_{o'}^\tau$.

3.4.4. Malicious gradient detection and correction for delayed clients

In asynchronous FL, the server may receive not only current gradients but also delayed gradients in each round of training. These delayed gradients may include malicious gradients sent by Byzantine clients, as well as conflicting gradients from benign clients that deviate from the current update direction. A straightforward approach would be to use MD to directly tackle these issues. However, based on the properties of MD described in Section 3.4.3, MD can only identify malicious gradients with significant anomalies in their numerical distribution; it cannot detect conflicting gradients that have a normal numerical distribution but deviate in their update direction. Therefore, we propose a malicious gradient detection and correction method by combining MD with projection correction, as shown in Algorithm 4, which mainly consists of two steps: first, MD is employed to detect obvious malicious gradients, yielding the initially filtered set of benign delayed gradients; second, projection correction is applied to the conflicting gradients in the delayed gradients that deviate from the current update direction.

Specifically, for the delayed gradients $\nabla W_d^\tau = (\nabla w_0^\tau, \nabla w_1^\tau, \dots, \nabla w_d^\tau)$ received by the server in the τ -th training round, we compute the MD value of each delayed gradient $\nabla w_j^\tau \in \nabla W_d^\tau$ relative to the benign current gradients $\nabla W_{o'}^\tau$, rather than to their own distribution. This is because the benign current gradients represent the correct update direction for the current training round, thus serving as the benchmark that the delayed gradients should align with. Furthermore, the computational overhead is substantially reduced by reusing the distribution parameters μ_{online} and Σ_{online} already computed in Section 3.4.3. Owing to the potentially wider dispersion of delayed gradients, a looser threshold than T_{online} is also required. Here we set the threshold as $T_{delayed} = T_{online} \cdot \alpha$, where α is the scaling factor that satisfies $\alpha > 1$. Consequently, delayed gradients with MD values greater than $T_{delayed}$ are identified as malicious, and we thus obtain the benign delayed gradients ∇W_{bd}^τ .

Moreover, since delayed gradients originate from previous training rounds, their update directions may conflict with those of current gradients, as analyzed in Section 3.3. To solve the potential conflict issue between the current gradient $\nabla w_i^\tau \in \nabla W_{o'}^\tau$ and the delayed gradient $\nabla w_{j \neq i}^\tau \in \nabla W_{bd}^\tau$ (i.e., $\nabla w_i^\tau \cdot \nabla w_j^\tau < 0$), we design a projection strategy to correct the delayed average gradient, enabling it to align with the current average gradient. Specifically, the server calculates the average of benign current gradients $\nabla W_{o'}^\tau$ (denoted as ∇w_m^τ), serving as the benchmark for this correction. Subsequently, the server employs the Deviation function (presented in Eq. (1)) to determine whether the delayed gradient $\nabla w_j^\tau \in \nabla W_{bd}^\tau$ conflicts with ∇w_m^τ . For all non-conflicting delayed gradients ∇w_j^τ that satisfy $\nabla w_m^\tau \cdot \nabla w_j^\tau > 0$, we calculate their average value $\nabla w_{m_1}^\tau$. For all conflicting delayed gradients ∇w_j^τ that satisfy $\nabla w_m^\tau \cdot \nabla w_j^\tau < 0$, we first calculate their average value $\nabla w_{m_2}^\tau$, and

Algorithm 4: Malicious Gradient Detection and Correction for Delayed Clients (MGDC).

Input: delayed gradients ∇W_d^τ , benign current gradients $\nabla W_{o'}^\tau$, current gradient parameters μ_{online} and Σ_{online} , threshold for current gradient T_{online} , scaling factor α

Output: Average of benign current gradients ∇w_m^τ , non-conflicting delayed gradients $\nabla w_{m_1}^\tau$, corrected conflicting delayed gradients $\nabla w_{m_2}^\tau$

- 1 Compute the average of benign current gradients:
 $\nabla w_m^\tau \leftarrow \text{mean}(\nabla W_{o'}^\tau)$
- 2 Set the delayed gradient detection threshold: $T_{delayed} = T_{online} \cdot \alpha$
- 3 Initialize empty sets $\nabla W_{nd}^\tau, \nabla W_{cd}^\tau$ // for categorizing benign delayed gradients
- 4 //Step 1: Malicious detection via MD
- 5 **for each** delayed gradient $\nabla w_j^\tau \in \nabla W_d^\tau$ **do**
- 6 Calculate its MD to the benign current gradient distribution:
 $MD(\nabla w_j^\tau) = \sqrt{(\nabla w_j^\tau - \mu_{online})^T \Sigma_{online}^{-1} (\nabla w_j^\tau - \mu_{online})}$
- 7 **if** $MD(\nabla w_j^\tau) \leq T_{delayed}$ **then**
- 8 //Step2: Categorize benign gradients
- 9 **if** $\nabla w_m^\tau \cdot \nabla w_j^\tau > 0$ **then**
- 10 Add ∇w_j^τ to ∇W_{nd}^τ //Non-conflicting delayed gradients
- 11 **end**
- 12 **else**
- 13 Add ∇w_j^τ to ∇W_{cd}^τ //Conflicting delayed gradients
- 14 **end**
- 15 **end**
- 16 **end**
- 17 Compute the average of non-conflicting gradients:
 $\nabla w_{m_1}^\tau \leftarrow \text{mean}(\nabla W_{nd}^\tau)$
- 18 Compute the average of conflicting gradients:
 $\nabla w_{m_2}^\tau \leftarrow \text{mean}(\nabla W_{cd}^\tau)$
- 19 Correct the average of conflicting gradients via projection:
 $\nabla w_{m_2}^\tau \leftarrow \nabla w_{m_2}^\tau - \frac{\nabla w_{m_2}^\tau \cdot \nabla w_m^\tau}{\|\nabla w_m^\tau\|^2} \nabla w_m^\tau$
- 20 **return** $\nabla w_m^\tau, \nabla w_{m_1}^\tau, \nabla w_{m_2}^\tau$

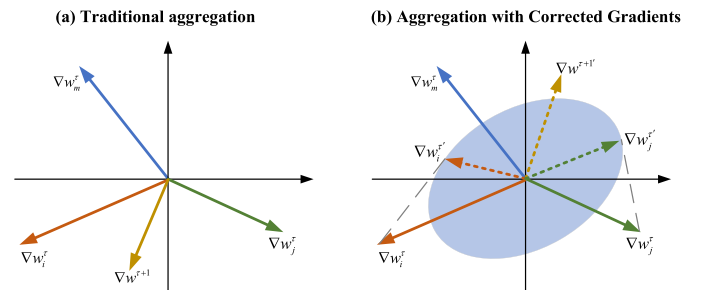


Fig. 2. Illustration of traditional aggregation and aggregation with corrected gradients.

then project $\nabla w_{m_2}^\tau$ onto the normal plane of ∇w_m^τ to remove inconsistent gradient components, as shown in Eq. (5).

$$\nabla w_{m_2}^\tau \leftarrow \nabla w_{m_2}^\tau - \frac{\nabla w_{m_2}^\tau \cdot \nabla w_m^\tau}{\|\nabla w_m^\tau\|^2} \nabla w_m^\tau \quad (5)$$

As shown in Fig. 2, ∇w_i^τ and ∇w_j^τ are two delayed gradients that conflict with the current average gradient ∇w_m^τ . It can be observed that

the update direction of $\nabla w^{\tau+1}$ aggregated from ∇w_i^τ and ∇w_j^τ deviates significantly from that of ∇w_m^τ , which in turn degrades the model performance. In contrast, we project the delayed gradients ∇w_i^τ and ∇w_j^τ onto the normal plane of ∇w_m^τ , and obtain the corrected gradients $\nabla w_i^{\tau'}$ and $\nabla w_j^{\tau'}$ respectively. It can be observed that the update direction of $\nabla w^{\tau+1}$ aggregated from $\nabla w_i^{\tau'}$ and $\nabla w_j^{\tau'}$ is more consistent with that of ∇w_m^τ , thus being more beneficial for the improvement of model performance.

3.4.5. Global model update

By aggregating the average of benign current gradients ∇w_m^τ , the average of non-conflicting delayed gradients $\nabla w_{m_1}^\tau$, and the average of corrected conflicting delayed gradients $\nabla w_{m_2}^\tau$, the global gradient can be computed as Eq. (6):

$$\nabla w^\tau = \alpha_0 \nabla w_m^\tau + \alpha_1 \nabla w_{m_1}^\tau + \alpha_2 \nabla w_{m_2}^\tau \quad (6)$$

where $\alpha_0, \alpha_1, \alpha_2$ are the weights assigned to ∇w_m^τ , $\nabla w_{m_1}^\tau$, and $\nabla w_{m_2}^\tau$, respectively, satisfying $\alpha_0 + \alpha_1 + \alpha_2 = 1$.

Based on this, the server updates the global model as shown in Eq. (7):

$$w^{\tau+1} \leftarrow w^\tau - \eta \nabla w^\tau \quad (7)$$

where η denotes the learning rate. Finally, the server distributes $w^{\tau+1}$ to the clients for the next communication round.

4. Convergence analysis

Assumption 1 (*L-smoothness*). For constant $L > 0$ and any w, w' , the function f is L -smooth if f has Lipschitz continuous gradients, i.e., for any w, w' ,

$$f(w) - f(w') \leq \langle \nabla f(w'), w - w' \rangle + \frac{L}{2} \|w - w'\|^2. \quad (8)$$

Assumption 2 (*Weak convexity*). A differentiable function f is μ -weakly convex if the function g with $g(w) = f(w) + \frac{\mu}{2} \|w\|^2$ is convex, where $\mu \geq 0$. f is convex if $\mu = 0$, and non-convex if $\mu > 0$.

Assumption 3 (*Bounded Delay*). For any delayed gradient $\nabla w_j^{\tau_j}$, the maximum delay rounds are bounded, i.e., there exists a constant K such that $\tau - \tau_j \leq K$.

Assumption 4 (*Gradient Noise Control*). For any $w \in \mathbb{R}^d$ and $d \sim D_i$, it holds that $\|\nabla g(w, d)\|^2 \leq V$.

Assumption 5 (*Controllable Residual Poisoned Gradients After Malicious Detection*). Following malicious detection of online and delayed gradients using the MD threshold, there exists a bounded deviation between the gradient mean and the true benign mean, such that

$$\mathbb{E} [\nabla w_m^\tau | w^\tau] = \nabla F(w^\tau) + b_\tau, \quad \|b_\tau\| \leq \rho_{on} \quad (9)$$

$$\mathbb{E} [\nabla w_{m'}^\tau | w^{\tau_j}] = \nabla F(w^{\tau_j}) + c_\tau, \quad \|c_\tau\| \leq \rho_{dl} \quad (10)$$

Here, ∇w_m^τ denotes the average of benign online gradients ∇W_o^τ , and $\nabla w_{m'}^\tau$ denotes the average of benign delayed gradients ∇W_d^τ . $\nabla F(w^\tau)$ and $\nabla F(w^{\tau_j})$ represent the global true gradients at the current round τ and the delayed round τ_j in the ideal case, respectively. b_τ and c_τ are the gradient deviations caused by false negatives or false positives in the malicious detection of online and delayed gradients, respectively. These deviations are bounded above by ρ_{on} and ρ_{dl} , which reflect the effectiveness of the proposed malicious detection method.

Assumption 6 (*Property of Projection Correction*). For conflicting delayed gradients, after being processed by the projection correction introduced in Section 3.4.4, they satisfy the following properties:

$$\langle \nabla w_{m_2}^\tau, \nabla w_{m'}^\tau \rangle = 0, \quad \|\nabla w_{m_2}^\tau\| \leq \|\nabla w_{m'}^\tau\| \quad (11)$$

Here, $\nabla w_{m_2}^\tau$ and $\nabla w_{m'}^\tau$ denote the averages of conflicting delayed gradients before and after correction, respectively, and ∇w_m^τ denotes the average of benign online gradients.

Lemma 1. Under Assumption 1, for any $\tau \geq 0$, the following inequality holds:

$$F(w^{\tau+1}) \leq F(w^\tau) - \eta \langle \nabla F(w^\tau), \nabla w^\tau \rangle + \frac{L\eta^2}{2} \|\nabla w^\tau\|^2 \quad (12)$$

Proof. By Assumption 1, we have

$$F(w^{\tau+1}) \leq F(w^\tau) + \langle \nabla F(w^\tau), w^{\tau+1} - w^\tau \rangle + \frac{L}{2} \|w^{\tau+1} - w^\tau\|^2$$

Substituting Eq. (7): $w^{\tau+1} = w^\tau - \eta \nabla w^\tau$, we obtain

$$F(w^{\tau+1}) \leq F(w^\tau) - \eta \langle \nabla F(w^\tau), \nabla w^\tau \rangle + \frac{L\eta^2}{2} \|\nabla w^\tau\|^2 \quad \square$$

Lemma 2. According to Eq. (6), where $\nabla w^\tau = \alpha_0 \nabla w_m^\tau + \alpha_1 \nabla w_{m_1}^\tau + \alpha_2 \nabla w_{m_2}^\tau$, the aggregated gradient ∇w^τ satisfies: $\|\nabla w^\tau\|^2 \leq V$.

Proof. By the convexity of vector norms and Jensen's inequality, we have:

$$\begin{aligned} \|\nabla w^\tau\|^2 &= \|\alpha_0 \nabla w_m^\tau + \alpha_1 \nabla w_{m_1}^\tau + \alpha_2 \nabla w_{m_2}^\tau\|^2 \\ &\leq \alpha_0 \|\nabla w_m^\tau\|^2 + \alpha_1 \|\nabla w_{m_1}^\tau\|^2 + \alpha_2 \|\nabla w_{m_2}^\tau\|^2 \\ &\leq \alpha_0 V + \alpha_1 V + \alpha_2 V = (\alpha_0 + \alpha_1 + \alpha_2) V = V \end{aligned} \quad \square$$

Lemma 3. Under Assumption 1 and 3, for any current round τ and delayed round τ_j , it holds that

$$\mathbb{E} \|\nabla F(w^\tau) - \nabla F(w^{\tau_j})\|^2 \leq L^2 \eta^2 K^2 V. \quad (13)$$

Proof. By using L -smoothness property and Eq. (7), we have

$$\|\nabla F(w^\tau) - \nabla F(w^{\tau_j})\| \leq L \|w^\tau - w^{\tau_j}\| \leq L \eta \sum_{i=\tau_j}^{\tau-1} \|\nabla w^i\|.$$

Combining with Lemma 2, we have

$$\begin{aligned} \mathbb{E} \|\nabla F(w^\tau) - \nabla F(w^{\tau_j})\|^2 &\leq L^2 \eta^2 \mathbb{E} \left\| \sum_{i=\tau_j}^{\tau-1} \nabla w^i \right\|^2 \\ &\leq L^2 \eta^2 (\tau - \tau_j) \sum_{i=\tau_j}^{\tau-1} \mathbb{E} \|\nabla w^i\|^2 \leq L^2 \eta^2 K \cdot K V = L^2 \eta^2 K^2 V. \end{aligned} \quad \square$$

Lemma 4 (*Benign Current Gradient*). Let ∇w_m^τ be the average of benign current gradients, then we have

$$\mathbb{E} [\langle \nabla F(w^\tau), \nabla w_m^\tau \rangle] \geq \frac{1}{2} \mathbb{E} \|\nabla F(w^\tau)\|^2 - \frac{1}{2} \rho_{on}^2. \quad (14)$$

Proof. Since $\nabla F(w^\tau)$ is a deterministic vector given the model parameter w^τ , we have

$$\mathbb{E} [\langle \nabla F(w^\tau), \nabla w_m^\tau \rangle | w^\tau] = \langle \nabla F(w^\tau), \mathbb{E} [\nabla w_m^\tau | w^\tau] \rangle \quad (15)$$

Substituting Eq. (9) into Eq. (15) yields

$$\begin{aligned} \mathbb{E} [\langle \nabla F(w^\tau), \nabla w_m^\tau \rangle | w^\tau] &= \langle \nabla F(w^\tau), \nabla F(w^\tau) + b_\tau \rangle \\ &= \|\nabla F(w^\tau)\|^2 + \langle \nabla F(w^\tau), b_\tau \rangle \\ &\geq \|\nabla F(w^\tau)\|^2 - \|\nabla F(w^\tau)\| \cdot \|b_\tau\| \\ &\geq \|\nabla F(w^\tau)\|^2 - \|\nabla F(w^\tau)\| \cdot \rho_{on} \\ &\geq \|\nabla F(w^\tau)\|^2 - \left(\frac{1}{2} \|\nabla F(w^\tau)\|^2 + \frac{1}{2} \rho_{on}^2 \right) \\ &= \frac{1}{2} \|\nabla F(w^\tau)\|^2 - \frac{1}{2} \rho_{on}^2. \end{aligned} \quad (16)$$

Taking the expectation on both sides of Eq. (16), we have

$$\begin{aligned} \mathbb{E} [\langle \nabla F(w^\tau), \nabla w_m^\tau \rangle] &= \mathbb{E} [\mathbb{E} [\langle \nabla F(w^\tau), \nabla w_m^\tau \rangle | w^\tau]] \\ &\geq \frac{1}{2} \mathbb{E} \|\nabla F(w^\tau)\|^2 - \frac{1}{2} \rho_{on}^2. \end{aligned} \quad \square$$

Lemma 5 (Non-Conflicting Delayed Gradient). Let $\nabla w_{m_1}^\tau$ be the average of non-conflicting delayed gradients, then we have

$$\mathbb{E} \left[\left\langle \nabla F(w^\tau), \nabla w_{m_1}^\tau \right\rangle \right] \geq \frac{1}{4} \mathbb{E} \|\nabla F(w^\tau)\|^2 - \frac{1}{2} L^2 \eta^2 K^2 V - \rho_{dl}^2. \quad (17)$$

Proof. Under [Assumption 5](#), we may assume that there exists a random vector c'_τ such that

$$\mathbb{E} \left[\nabla w_{m_1}^\tau \mid w^{\tau_j} \right] = \nabla F(w^{\tau_j}) + c'_\tau, \quad \|c'_\tau\| \leq \rho_{dl}. \quad (18)$$

Then we have

$$\mathbb{E} \left[\left\langle \nabla F(w^\tau), \nabla w_{m_1}^\tau \right\rangle \right] = \langle \nabla F(w^\tau), \nabla F(w^{\tau_j}) \rangle + \langle \nabla F(w^\tau), c'_\tau \rangle. \quad (19)$$

For the first term $\langle \nabla F(w^\tau), \nabla F(w^{\tau_j}) \rangle$ in [Eq. \(19\)](#), by the identity

$$\langle a, b \rangle = \frac{1}{2} \|a\|^2 + \frac{1}{2} \|b\|^2 - \frac{1}{2} \|a - b\|^2 \geq \frac{1}{2} \|a\|^2 - \frac{1}{2} \|a - b\|^2,$$

we obtain

$$\langle \nabla F(w^\tau), \nabla F(w^{\tau_j}) \rangle \geq \frac{1}{2} \|\nabla F(w^\tau)\|^2 - \frac{1}{2} \|\nabla F(w^\tau) - \nabla F(w^{\tau_j})\|^2. \quad (20)$$

For the second term $\langle \nabla F(w^\tau), c'_\tau \rangle$ in [Eq. \(19\)](#), by the Cauchy-Schwarz inequality, Young's inequality, and the condition $\|c'_\tau\| \leq \rho_{dl}$, we have

$$\begin{aligned} \langle \nabla F(w^\tau), c'_\tau \rangle &\geq -\|\nabla F(w^\tau)\| \|c'_\tau\| \geq -\rho_{dl} \|\nabla F(w^\tau)\| \\ &\geq -\frac{1}{4} \|\nabla F(w^\tau)\|^2 - \rho_{dl}^2. \end{aligned} \quad (21)$$

Substituting [Eqs. \(20\) and \(21\)](#) into [Eq. \(19\)](#), together with [Lemma 3](#), we have

$$\begin{aligned} \mathbb{E} \left[\left\langle \nabla F(w^\tau), \nabla w_{m_1}^\tau \right\rangle \right] &\geq \frac{1}{4} \|\nabla F(w^\tau)\|^2 - \frac{1}{2} \|\nabla F(w^\tau) - \nabla F(w^{\tau_j})\|^2 - \rho_{dl}^2 \\ &\geq \frac{1}{4} \|\nabla F(w^\tau)\|^2 - \frac{1}{2} L^2 \eta^2 K^2 V - \rho_{dl}^2. \end{aligned} \quad \square$$

Lemma 6 (Corrected Conflicting Delayed Gradient). Let $\nabla w_{m_2}^\tau$ be the average of corrected conflicting delayed gradients, then we have

$$\mathbb{E} \left[\left\langle \nabla F(w^\tau), \nabla w_{m_2}^\tau \right\rangle \right] \geq -\frac{1}{2} \rho_{on}^2 - \frac{1}{2} \|\nabla w_{m_2}^\tau\|^2 \quad (22)$$

Proof. Based on [Eq. \(9\)](#) and [Eq. \(11\)](#), we have

$$\begin{aligned} \mathbb{E} \left[\left\langle \nabla F(w^\tau), \nabla w_{m_2}^\tau \right\rangle \right] &= \mathbb{E} \left[\left\langle \nabla w_m^\tau - b_\tau, \nabla w_{m_2}^\tau \right\rangle \right] \\ &= \mathbb{E} \left[\left\langle \nabla w_m^\tau, \nabla w_{m_2}^\tau \right\rangle \right] - \mathbb{E} \left[\left\langle b_\tau, \nabla w_{m_2}^\tau \right\rangle \right] = -\mathbb{E} \left[\left\langle b_\tau, \nabla w_{m_2}^\tau \right\rangle \right] \\ &\geq -\|b_\tau\| \cdot \mathbb{E} \|\nabla w_{m_2}^\tau\| \geq -\rho_{on} \cdot \mathbb{E} \|\nabla w_{m_2}^\tau\| \\ &\geq -\frac{1}{2} \rho_{on}^2 - \frac{1}{2} \mathbb{E} \|\nabla w_{m_2}^\tau\|^2 \end{aligned} \quad \square$$

Theorem 1. Under [Assumptions 1–6](#), choose the learning rate $\eta \leq \frac{1}{L}$, and let $A = \frac{\alpha_0 + \alpha_2}{2} \rho_{on}^2 + \alpha_1 \rho_{dl}^2$, $B = \frac{\alpha_1}{2} L^2 \eta^2 K^2 V + \frac{\alpha_2}{2} V$, $C = \frac{\alpha_0}{2} + \frac{\alpha_1}{4}$. If $F^* = \inf_w F(w)$, after T global training rounds, [Algorithm 1](#) converges to a critical point:

$$\min_{\tau=0}^{T-1} \mathbb{E} \|\nabla F(w^\tau)\|^2 \leq \frac{F(w^0) - F^*}{\eta CT} + \frac{1}{C} \left(A + B + \frac{L\eta}{2} V \right).$$

Proof. From [Lemma 1](#), we have

$$\mathbb{E} [F(w^{\tau+1})] \leq \mathbb{E} [F(w^\tau)] - \eta \mathbb{E} \langle \nabla F(w^\tau), \nabla w^\tau \rangle + \frac{L\eta^2}{2} \mathbb{E} \|\nabla w^\tau\|^2. \quad (23)$$

Based on [Eq. \(6\)](#), we have

$$\begin{aligned} \mathbb{E} \langle \nabla F(w^\tau), \nabla w^\tau \rangle &= \alpha_0 \mathbb{E} \langle \nabla F(w^\tau), \nabla w_m^\tau \rangle + \\ &\alpha_1 \mathbb{E} \langle \nabla F(w^\tau), \nabla w_{m_1}^\tau \rangle + \alpha_2 \mathbb{E} \langle \nabla F(w^\tau), \nabla w_{m_2}^\tau \rangle \end{aligned} \quad (24)$$

Substituting [Eqs. \(14\), \(17\), and \(22\)](#) into [Eq. \(24\)](#), and combining with $\mathbb{E} \|\nabla w_{m_2}^\tau\|^2 \leq V$ (derived from [Assumption 4](#)), we obtain

$$\begin{aligned} \mathbb{E} \langle \nabla F(w^\tau), \nabla w^\tau \rangle &\geq \underbrace{\left(\frac{\alpha_0}{2} + \frac{\alpha_1}{4} \right)}_{=: C} \mathbb{E} \|\nabla F(w^\tau)\|^2 - \\ &\underbrace{\left(\frac{\alpha_0 + \alpha_2}{2} \rho_{on}^2 + \alpha_1 \rho_{dl}^2 \right)}_{=: A} - \underbrace{\left(\frac{\alpha_1}{2} L^2 \eta^2 K^2 V + \frac{\alpha_2}{2} V \right)}_{=: B}. \end{aligned} \quad (25)$$

Substituting [Eq. \(25\)](#) back into [Eq. \(23\)](#) yields

$$\begin{aligned} \mathbb{E} [F(w^{\tau+1})] &\leq \mathbb{E} [F(w^\tau)] - \eta C \mathbb{E} \|\nabla F(w^\tau)\|^2 + \eta A + \eta B + \frac{L\eta^2}{2} \mathbb{E} \|\nabla w^\tau\|^2 \\ &\leq \mathbb{E} [F(w^\tau)] - \eta C \mathbb{E} \|\nabla F(w^\tau)\|^2 + \eta A + \eta B + \frac{L\eta^2}{2} V. \end{aligned} \quad (26)$$

Summing [Eq. \(26\)](#) from $\tau = 0$ to $\tau = T - 1$, we get

$$\begin{aligned} \sum_{\tau=0}^{T-1} (\mathbb{E} [F(w^{\tau+1})] - \mathbb{E} [F(w^\tau)]) &= \mathbb{E} [F(w^T)] - F(w^0) \\ &\leq -\eta C \sum_{\tau=0}^{T-1} \mathbb{E} \|\nabla F(w^\tau)\|^2 + T\eta A + T\eta B + T \frac{L\eta^2}{2} V. \end{aligned}$$

Since the lower bound $F(w^T) \geq F^* = \inf_w F(w)$ holds, we have

$$F^* - F(w^0) \leq -\eta C \sum_{\tau=0}^{T-1} \mathbb{E} \|\nabla F(w^\tau)\|^2 + T\eta A + T\eta B + T \frac{L\eta^2}{2} V.$$

Rearranging terms and dividing both sides by ηCT yields

$$\frac{1}{T} \sum_{\tau=0}^{T-1} \mathbb{E} \|\nabla F(w^\tau)\|^2 \leq \frac{F(w^0) - F^*}{\eta CT} + \frac{1}{C} \left(A + B + \frac{L\eta}{2} V \right).$$

Therefore, after completing T global training rounds, we have

$$\begin{aligned} \min_{\tau=0}^{T-1} \mathbb{E} \|\nabla F(w^\tau)\|^2 &\leq \frac{1}{T} \sum_{\tau=0}^{T-1} \mathbb{E} \|\nabla F(w^\tau)\|^2 \\ &\leq \frac{F(w^0) - F^*}{\eta CT} + \frac{1}{C} \left(A + B + \frac{L\eta}{2} V \right) \end{aligned} \quad \square$$

Theorem 1 shows that the convergence is affected by three major factors.

- (i) *Staleness of delayed gradient*: the bounded delay K appears in B as $\frac{\alpha_1}{2} L^2 \eta^2 K^2 V$. Hence larger staleness leads to a looser upper bound and slows down convergence, and its impact can be mitigated by assigning a smaller weight w_2 to delayed gradients.
- (ii) *Malicious gradient detection*: the residual bias after MD filtering contributes to the term $A = \frac{\alpha_0 + \alpha_2}{2} \rho_{on}^2 + \alpha_1 \rho_{dl}^2$, which does not vanish with T . Therefore, imperfect detection yields a non-zero error floor, while improving detection accuracy reduces ρ_{on}, ρ_{dl} and hence tightens the bound.
- (iii) *Non-IID data*: Non-IID data increases the second moment bound V in [Assumption 4](#), which enlarges both the smoothness-related term $\frac{L\eta}{2} V$ and the staleness term $\frac{\alpha_1}{2} L^2 \eta^2 K^2 V$. This indicates that non-IID data not only degrades convergence itself but also amplifies the negative effect of staleness.

5. Empirical evaluation

In this section, we evaluate the performance of BRAFed and compare it with several state-of-the-art counterparts.

5.1. Experimental setup

5.1.1. Datasets and models

Three real-world datasets are employed in the experiments, representing data of varying complexities and modalities that can exist in IoT systems:

1. CIFAR-100 dataset contains 50,000 training samples and 10,000 test samples, each of which is a 32×32 pixel color image. For CIFAR-100, we use the widely used ResNet18 architecture [37] as the global model.
2. Shakespeare [1] is a large-scale NLP dataset derived from the plays of William Shakespeare, containing approximately 4.3 million character-level sequences from 1129 distinct play characters. We choose a two-layer LSTM classifier as the basic experimental model, which contains 100 hidden units with an 8D embedding layer. The model takes a sequence of 80 characters as input, embeds each character into a learned 8-dimensional space, and outputs one character for each training sample after 2 LSTM layers and a densely connected layer.
3. COVIDx [38] is a large-scale, multinational chest radiograph dataset, containing over 30,000 images from 16,400 patients. Each image is a grayscale chest X-ray of varying resolution, pre-processed to 224×224 pixels for input to deep learning models. For COVIDx, we use the widely used ResNet50 architecture [39] as the global model.

5.1.2. Attack settings

Six attack scenarios are considered in this paper, two for data poisoning attacks and four for model poisoning attacks, which are as follows:

1. Backdoor (BD) attack [40]: BD attack is a targeted data poisoning attack. The attacker injects a predefined trigger into part of its local training samples and changes their labels to an attacker-specified target class. For CIFAR-100 and COVIDx, the trigger is implemented as a fixed patch pattern attached to the input images; for Shakespeare, the trigger is implemented as a fixed token/character pattern inserted into the input sequence.
2. Label flipping (LF) attack [23]: LF attack is a data poisoning attack. We convert the source label s of its local sample to the target label $l - s - 1$, where l is the total number of labels in the dataset.
3. Gaussian (GS) attack [22]: GS attack is a model poisoning attack. We replace the raw gradient with random noise sampled from a zero-mean Gaussian distribution (with the standard deviation of the noise set to 200).
4. Model replacement (MR) attack [41]: The backdoor trigger of MR attack is set up as for backdoor model poisoning attack. Following the work [41], in order to mitigate the dilution of the attack effect due to global aggregation, we extend the poisoning local gradients by setting the weight factor $\gamma = n$ and send the extended local gradients to the server, where n is the total number of clients.
5. LIE attack [42]: LIE attack is a typical colluding model poisoning attack. The attacker manipulates all malicious clients to provide a range of perturbations. The attacker uses the maximum value $z^{\max} \leftarrow \max_z \left(\Phi(z) < \frac{N-M-s}{N-M} \right)$ to circumvent the defense.
6. Adaptive attack: It is a powerful model poisoning attack, built on the state-of-the-art adaptive attack framework [43] and tailored to BRAFed's online gradient detection mechanism. We assume that the attacker has full knowledge of the defense rule deployed on the server and can optimize malicious gradients accordingly. For simplicity, we focus only on attacking the current gradients, while leaving delayed gradients unchanged. Let the online gradients at round τ be

$$\nabla W_o^\tau = \{\nabla w_1^\tau, \nabla w_2^\tau, \dots, \nabla w_p^\tau\}, \quad (27)$$

where the first c clients are Byzantine clients and the remaining $p-c$ clients are benign ones. The attacker aims to maximize the deviation of the attacked online aggregation from the benign online aggregation along the opposite direction of the benign gradient, while ensuring that the crafted malicious gradients can evade the MGDO detector. Denote the average benign online gradient and its normalized direction by

$$\bar{g}_h^\tau = \frac{1}{p-c} \sum_{i=c+1}^p \nabla w_i^\tau, \quad s^\tau = \frac{\bar{g}_h^\tau}{\|\bar{g}_h^\tau\|}. \quad (28)$$

Meanwhile, the benign online mean and covariance are estimated as

$$\mu_h^\tau = \frac{1}{p-c} \sum_{i=c+1}^p \nabla w_i^\tau, \quad \Sigma_h^\tau = \text{Cov}\left(\left\{\nabla w_{c+1}^\tau, \dots, \nabla w_p^\tau\right\}\right). \quad (29)$$

Accordingly, the adaptive attack is formulated as

$$\max_{\nabla w_1^{\tau*}, \dots, \nabla w_c^{\tau*}} (s^\tau)^T (\bar{g}_h^\tau - \bar{g}_{\text{adv}}^\tau) \quad (30)$$

subject to

$$\bar{g}_{\text{adv}}^\tau = \frac{1}{p} \left(\sum_{i=1}^c \nabla w_i^{\tau*} + \sum_{i=c+1}^p \nabla w_i^\tau \right), \quad (31)$$

$$MD(\nabla w_i^{\tau*}; \mu_h^\tau, \Sigma_h^\tau) \leq T_{\text{online}}^\tau, \quad i = 1, \dots, c, \quad (32)$$

Under the above approximation, the MD constraint defines an ellipsoidal feasible region, and the strongest feasible malicious gradient is achieved on its boundary. Thus, all Byzantine online clients upload the same crafted gradient:

$$\nabla w_{\text{adv}}^\tau = \mu_h^\tau - T_{\text{online}}^\tau \frac{\Sigma_h^\tau s^\tau}{\sqrt{(s^\tau)^T \Sigma_h^\tau s^\tau}}. \quad (33)$$

Therefore, the attacker sets

$$\nabla w_1^{\tau*} = \dots = \nabla w_c^{\tau*} = \nabla w_{\text{adv}}^\tau. \quad (34)$$

5.1.3. Parameter settings

Our evaluation is conducted on an AMD EPYC 7453 CPU, 64 GB DDR4 memory, and an RTX 4090 GPU with 48 GB VRAM, operating on Ubuntu 20.04 with PyTorch and Python 3.9.

The Non-IID setting of FL is simulated using the Dirichlet distribution, denoted as $\text{Dir}(n, \beta)$, where n is the number of clients and β controls the degree of Non-IIDness in the dataset. A smaller β corresponds to a higher degree of Non-IIDness. Among all clients, delayed clients account for a proportion δ . To simulate a more realistic delay environment, we use the Pareto distribution $\text{Pareto}(x_m = 3, \alpha = 1.5)$. This distribution models a scenario where the majority of clients have low delays, while a small number of clients experience significantly higher delays. In our experiment, the majority of clients' delays are concentrated between 3 and 10 rounds, while the delays of a few clients reach 15 or even higher. This setting aligns with typical FL scenarios, where most clients have fast or moderate network conditions, but a small subset of clients may experience large delays due to factors such as network instability or resource constraints.

Additionally, the experimental parameters used in this study are listed in Table 2, including the global learning rate η , number of clients n , total training rounds T , Non-IID degree β , local epochs E , local batch size B , delay ratio δ , attack ratio σ , weights in Eq. (6) $\alpha_0, \alpha_1, \alpha_2$, sensitivity coefficient k , decay factor λ , threshold scaling factor α , and the hyperparameter in Eq. (2) ϵ . Unless otherwise stated, the parameter values in Table 2 are the defaults, which represent the optimal settings obtained through multiple parameter tuning iterations. The experimental results reported for this method are averaged over five independent runs.

Table 2
Parameter setting.

Parameter	Value	Parameter	Value
η	0.01	α_0	0.6
n	100	α_1	0.25
β	0.1	α_2	0.15
T	100 for Shakespeare 200 for CIFAR-100 and COVIDx	λ	0.04 for Shakespeare 0.01 for CIFAR-100 and COVIDx
E	1 for Shakespeare 5 for CIFAR-100 and COVIDx	B	10 for Shakespeare 64 for CIFAR-100 and COVIDx
δ	40%	α	1.2
σ	30%	k	3
ε	0.1		

5.1.4. Benchmarks

We compare BRAFed with the following representative baselines:

1. FedAvg [1]: the standard synchronous FL method based on weighted averaging.
2. AFLGuard [18]: an asynchronous Byzantine-robust FL method that relies on a small trusted dataset to filter malicious updates.
3. FedCPA [30]: a Byzantine-robust FL method for Non-IID data that detects malicious clients via critical parameter similarity.
4. FedSSuper [34]: a recent robust FL framework for Non-IID data that uses a public shadow dataset and dual filtering to remove malicious updates.

5.1.5. Evaluation metrics

To comprehensively evaluate the performance and robustness of BRAFed, we adopt five evaluation metrics:

1. **Overall Accuracy (ACC)**: the fraction of testing examples that are correctly classified. Accuracy reflects the general performance of the trained global model across all classes, with higher values indicating better overall predictive capability.
2. **Clean Accuracy (CA)**: the accuracy of the model on test samples under non-attack conditions.
3. **Robust Accuracy (RA)**: the average ACC of the model under six attack scenarios, including BD, LF, GS, MR, LIE and adaptive attacks.
4. **Precision rate (PR)**: the proportion of correctly predicted malicious instances to the total number of predicted malicious.
5. **Recall rate (RR)**: the proportion of correctly predicted malicious instances to all malicious.
6. **Attack Success Rate (ASR)**: the fraction of trigger-embedded testing inputs that are classified as the attacker-chosen target label. ASR measures the effectiveness of poisoning attacks, with lower values indicating stronger resilience of the defense. Note that ASR is only applicable for targeted attacks, such as the BD attack and model-replacement attack considered in our experiments.

5.2. Experimental results

5.2.1. Sensitivity analysis of hyperparameters

Choosing k and λ . As discussed in Section 3.4.3, the choice of k and λ has an impact on BRAFed. k controls the strictness of malicious detection: a larger k corresponds to a looser threshold, which may lead to more malicious gradients being misclassified as benign; conversely, a smaller k indicates a stricter threshold, which may mistakenly identify some benign gradients as malicious. Meanwhile, λ governs the rate at which the threshold decreases. Given the distinct characteristics of each dataset, we explore the impact of various combinations of k and λ by heuristically selecting values from $k \in \{2, 3, 4\}$ and

Table 3

Effect of hyper-parameters k and λ for CIFAR-100, Shakespeare and COVIDx. The empirically chosen trade-off between CA and RA is in bold.

Dataset	k	$\lambda = 0.01$		$\lambda = 0.02$		$\lambda = 0.03$		$\lambda = 0.04$	
		CA	RA	CA	RA	CA	RA	CA	RA
CIFAR-100	2	0.322	0.306	0.332	0.299	0.331	0.289	0.332	0.282
	3	0.338	0.315	0.337	0.312	0.335	0.308	0.334	0.305
	4	0.328	0.288	0.333	0.277	0.332	0.261	0.331	0.254
Shakespeare	2	0.468	0.415	0.474	0.422	0.472	0.421	0.474	0.429
	3	0.476	0.433	0.480	0.438	0.478	0.436	0.482	0.441
	4	0.471	0.402	0.475	0.409	0.473	0.413	0.477	0.424
COVIDx	2	0.836	0.808	0.833	0.807	0.833	0.806	0.837	0.801
	3	0.841	0.813	0.838	0.809	0.835	0.811	0.837	0.808
	4	0.838	0.802	0.834	0.798	0.836	0.800	0.834	0.795

Table 4

Effect of hyper-parameters α for CIFAR-100, Shakespeare, and COVIDx. The empirically chosen trade-off between CA and RA is in bold.

Dataset	α	$\tau_{stale} = 4$		$\tau_{stale} = 8$		$\tau_{stale} = 12$		$\tau_{stale} = 16$	
		CA	RA	CA	RA	CA	RA	CA	RA
CIFAR-100	1.1	0.339	0.314	0.333	0.312	0.332	0.309	0.326	0.296
	1.2	0.342	0.320	0.338	0.315	0.336	0.312	0.331	0.308
	1.3	0.340	0.314	0.335	0.305	0.334	0.304	0.328	0.304
	1.4	0.337	0.301	0.334	0.297	0.329	0.295	0.325	0.288
Shakespeare	1.1	0.482	0.432	0.480	0.436	0.477	0.430	0.471	0.422
	1.2	0.486	0.439	0.482	0.441	0.480	0.437	0.475	0.431
	1.3	0.480	0.436	0.478	0.437	0.474	0.434	0.476	0.433
	1.4	0.478	0.421	0.474	0.428	0.470	0.426	0.468	0.415
COVIDx	1.1	0.839	0.808	0.837	0.806	0.834	0.805	0.830	0.802
	1.2	0.843	0.815	0.841	0.813	0.838	0.810	0.834	0.805
	1.3	0.842	0.813	0.839	0.810	0.836	0.809	0.832	0.808
	1.4	0.835	0.803	0.835	0.801	0.834	0.797	0.833	0.796

$\lambda \in \{0.01, 0.02, 0.03, 0.04\}$ for different datasets. We empirically choose the best trade-off between CA and RA. The results for CIFAR-100, Shakespeare and COVIDx are reported in Table 3. Empirically, we find that the best trade-off combination for CIFAR-100 and COVIDx is $k = 3$ and $\lambda = 0.01$. For Shakespeare, the trade-off combination is $k = 3$ and $\lambda = 0.04$. In practice, we suggest tuning the hyperparameters as follows. The sensitivity coefficient k is recommended to be within the range of [2, 4], and the decay factor λ should be selected from [0.01, 0.04]. When the proportion of malicious clients increases, a larger k is preferable to maintain a reasonable threshold. Conversely, when the data is more heterogeneous or the model requires longer training to converge, a smaller λ is advised to allow the threshold to decrease more gradually. These guidelines provide a practical reference for setting the Mahalanobis distance threshold parameters in real-world FL scenarios.

Choosing α . As discussed in Section 3.4.4, the choice of α determines the scaling factor applied to the MD threshold for delayed gradients relative to the threshold for current gradients. In theory, the staler a delayed gradient is, the larger its MD with respect to the distribution of benign current gradients, suggesting that α should be set proportionally to the staleness. We denote the average staleness of all delayed gradients as τ_{stale} , defined as the number of rounds by which they lag behind the current training progress, which can be experimentally controlled. Given the distinct characteristics of each dataset, we explore the impact of various combinations of α and τ_{stale} by heuristically selecting values from $\alpha \in \{1.1, 1.2, 1.3, 1.4\}$ and $\tau_{stale} \in \{4, 8, 12, 16\}$ for different datasets. We empirically determine the best trade-off between CA and RA. The results for CIFAR-100, Shakespeare, and COVIDx are reported in Table 4. Empirically, when $\tau_{stale} \leq 12$, the optimal value of α across all three datasets is 1.2; when $\tau_{stale} = 16$, the optimal α is 1.3 for Shakespeare and COVIDx, and 1.2 for CIFAR-100. Since the overall staleness of delayed gradients is generally unpredictable in practice, we recommend

choosing α within the range [1.2, 1.3]. For the experiments reported in this work, we set $\alpha = 1.2$.

Choosing $\alpha_0, \alpha_1, \alpha_2$. As discussed in Section 3.4.5, the hyperparameters α_0, α_1 , and α_2 correspond to the weights assigned to the benign current gradients, non-conflicting delayed gradients, and corrected conflicting delayed gradients, respectively. In principle, these three hyperparameters should satisfy $\alpha_0 + \alpha_1 + \alpha_2 = 1$, reflecting their relative contributions to the final aggregated gradient. Given the distinct characteristics of each dataset, we investigate the impact of different combinations of α_0 and α_1 by heuristically selecting values from $\alpha_0 \in \{0.5, 0.6, 0.7\}$ and $\alpha_1 \in \{0.2, 0.25, 0.3, 0.35, 0.4\}$. The remaining weight α_2 is set as $\alpha_2 = 1 - \alpha_0 - \alpha_1$. We empirically determine the best trade-off between CA and RA. The results for CIFAR-100, Shakespeare, and COVIDx are reported in Table 5. Empirically, we observe that $\alpha_0 = 0.6$, $\alpha_1 = 0.25$, and $\alpha_2 = 0.15$ achieve the highest accuracy across all three datasets. Therefore, these values are adopted as the default setting in this work.

5.2.2. Effectiveness of projection strategies

In Section 3.4.4, the projection strategy removes the conflicting components of delayed gradients relative to the current average gradient, ensuring that the update directions of the model are not adversely affected by gradient staleness. To empirically investigate whether such projection may inadvertently suppress useful complementary information in delayed gradients or affect client-level fairness, we design four projection strategy variants for the experiments. Specifically, we introduce a parameter γ to control the projection strength: $\gamma = 1$ corresponds to full projection (as in the proposed BRAFed method), $\gamma = 0$ corresponds to no projection, and intermediate values $0 < \gamma < 1$ implement partial projection, removing only a fraction γ of the conflicting component. In our experiments, we consider $\gamma = 0, 0.5, 0.8$, and 1.

We evaluate these four strategies under three levels of data heterogeneity, represented by Non-IID parameters $\beta = 0.1, 0.5, 1$, across the CIFAR-100, Shakespeare and COVIDx datasets. Table 6 reports the CA and RA for each strategy and heterogeneity setting. The No Projection variant ($\gamma = 0$) yields the lowest RA and the lowest CA, indicating that directly employing conflicting delayed gradients impairs model accuracy. The Full Projection variant (BRAFed, $\gamma = 1$) exhibits slightly lower CA compared to the Scaled Projection strategy with $\gamma = 0.8$, indicating that the projection strategy may remove some useful complementary information contained in delayed gradients. Nevertheless, the Full Projection variant consistently achieves the highest RA across all datasets and Non-IID levels, demonstrating that BRAFed is particularly effective in scenarios involving poisoning attacks.

To assess client-level fairness, we visualize the RA distributions for all clients under different projection strategies and Non-IID levels using boxplots (Fig. 3). Each box represents the distribution of RA among clients for a given strategy and β . Longer boxes indicate greater disparity among client performance. A general trend observed from the boxplots is that increasing the projection strength γ leads to higher RA

Table 6

Performance of different projection strategies under varying data heterogeneity. The empirically chosen trade-off between CA and RA is in bold.

Dataset	γ	$\beta = 0.1$		$\beta = 0.5$		$\beta = 1$	
		CA	RA	CA	RA	CA	RA
CIFAR-100	0	0.312	0.280	0.323	0.288	0.326	0.295
	0.5	0.327	0.296	0.336	0.303	0.343	0.307
	0.8	0.343	0.311	0.345	0.312	0.354	0.315
	1	0.338	0.315	0.343	0.317	0.352	0.322
Shakespeare	0	0.455	0.393	0.460	0.396	0.466	0.415
	0.5	0.478	0.412	0.481	0.415	0.485	0.433
	0.8	0.489	0.430	0.494	0.431	0.490	0.442
	1	0.482	0.441	0.484	0.443	0.491	0.452
COVIDx	0	0.818	0.772	0.822	0.781	0.825	0.787
	0.5	0.832	0.792	0.835	0.800	0.836	0.803
	0.8	0.842	0.805	0.844	0.810	0.845	0.812
	1	0.841	0.813	0.843	0.815	0.847	0.816

and shorter boxes, indicating that stronger projection not only improves model accuracy under poisoning attacks but also reduces the performance differences between clients. An exception is the Full Projection variant (BRAFed, $\gamma = 1$), whose box is slightly longer than that of the Scaled Projection ($\gamma = 0.8$), suggesting that the projection strategy may slightly increase the performance gap between a few underrepresented clients and dominant clients. Nevertheless, the Full Projection variant consistently achieves the highest median RA, demonstrating that the projection strategy attains a balanced trade-off between overall model accuracy and client-level fairness.

5.2.3. Performance on defense against poisoning attacks

In this section, we employ the default parameter setting, with the Non-IID degree $\beta = 0.1$ and a delay ratio of 40%. Table 7 presents the detection results (PR and RR) of four defense methods across different attack types and three benchmark datasets. BRAFed consistently outperforms the other defense methods, achieving the highest PR and RR in most scenarios. In particular, BRAFed attains 100% detection accuracy for LF, GS, and LIE attacks on both the CIFAR-100 and Shakespeare datasets.

Although FedCPA and FedSSuper achieve detection performance comparable to BRAFed for BD, LF, and GS attacks, their effectiveness deteriorates on MR and LIE attacks. For instance, on the more challenging COVIDx dataset, BRAFed achieves PR and RR values for MR attacks that are 70% and 74% higher than those of FedCPA, and 46% and 30% higher than those of FedSSuper, respectively. For stronger adaptive attacks, FedCPA exhibits the poorest performance, whereas FedSSuper ranks second only to BRAFed. This is attributable to FedSSuper's lack of consideration for delayed gradients, which can result in partially delayed gradients being misclassified as malicious. AFLGuard performs worse than the other three defense methods across most attack types, rendering it unsuitable for highly heterogeneous scenarios with $\beta = 0.1$.

Table 5

Effect of hyper-parameters $\alpha_0, \alpha_1, \alpha_2$ for CIFAR-100, Shakespeare, and COVIDx. The empirically chosen trade-off between CA and RA is in bold.

Dataset	α_0	$\alpha_1 = 0.2$		$\alpha_1 = 0.25$		$\alpha_1 = 0.3$		$\alpha_1 = 0.35$		$\alpha_1 = 0.4$	
		CA	RA	CA	RA	CA	RA	CA	RA	CA	RA
CIFAR-100	0.5	0.321	0.303	0.329	0.314	0.324	0.306	0.330	0.305	0.325	0.300
	0.6	0.334	0.310	0.338	0.315	0.335	0.311	0.326	0.302	0.319	0.292
	0.7	0.331	0.309	0.332	0.309	0.329	0.301	—	—	—	—
Shakespeare	0.5	0.473	0.420	0.481	0.431	0.478	0.431	0.479	0.430	0.467	0.425
	0.6	0.480	0.432	0.482	0.441	0.474	0.425	0.470	0.425	0.463	0.421
	0.7	0.476	0.425	0.474	0.428	0.467	0.421	—	—	—	—
COVIDx	0.5	0.832	0.797	0.831	0.800	0.827	0.803	0.824	0.808	0.823	0.802
	0.6	0.838	0.803	0.841	0.813	0.836	0.809	0.835	0.805	0.832	0.806
	0.7	0.835	0.801	0.836	0.803	0.830	0.804	—	—	—	—

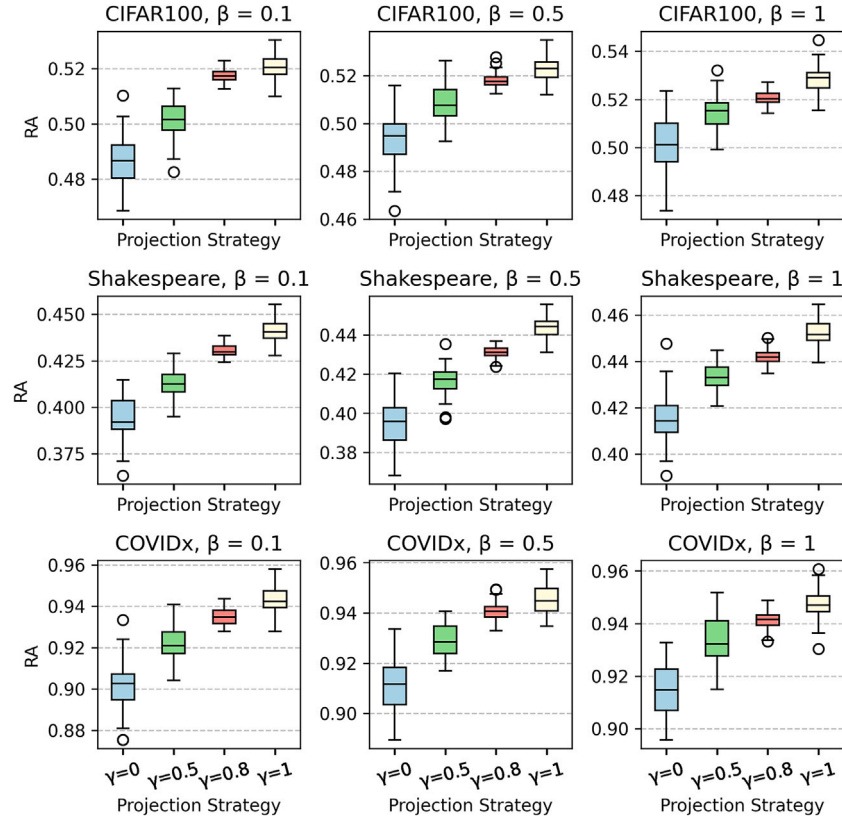


Fig. 3. Client-level RA under different projection strategies and data heterogeneity.

Table 7

Results (PR and RR) of malicious client detection for different attacks, and detection methods under three datasets. The best results are bolded.

Dataset	Method	BD attack		LF attack		GS attack		MR attack		LIE attack		Adaptive attack	
		PR	RR	PR	RR	PR	RR	PR	RR	PR	RR	PR	RR
CIFAR-100	AFLGuard	0.83	0.82	0.85	0.88	0.86	0.84	0.45	0.39	0.46	0.54	0.64	0.62
	FedCPA	0.96	0.99	1	1	0.92	1	0.38	0.26	0.08	0.1	0.25	0.46
	FedSSuper	0.93	0.98	0.93	0.95	1	1	0.47	0.72	0.53	0.76	0.73	0.85
	BRAFed	0.95	0.96	1	1	1	1	0.93	0.97	1	1	0.81	0.94
Shakespeare	AFLGuard	0.78	0.82	0.83	0.86	0.82	0.82	0.56	0.64	0.03	0.04	0.75	0.72
	FedCPA	0.92	1	0.92	0.99	1	1	0.44	0.26	0.31	0.51	0.29	0.5
	FedSSuper	0.96	0.98	0.98	1	0.98	0.94	0.56	0.75	0.44	0.65	0.88	0.92
	BRAFed	0.985	0.98	1	1	1	1	0.92	0.95	1	1	0.91	0.93
COVIDx	AFLGuard	0.88	0.85	0.80	0.76	0.83	0.88	0.12	0.24	0.06	0.07	0.61	0.70
	FedCPA	0.93	0.95	0.84	0.88	0.9	0.94	0.18	0.22	0.21	0.18	0.22	0.14
	FedSSuper	0.92	0.94	0.89	0.82	0.94	0.93	0.42	0.66	0.36	0.69	0.69	0.78
	BRAFed	0.96	0.98	0.96	0.85	0.97	1	0.88	0.96	0.83	0.94	0.74	0.82

Table 8 also reports the ACC and ASR of the global model for the three datasets under different attacks. Since BD, MR, and LIE attacks are targeted, the ASR of each defense method is evaluated specifically against these attacks. Apart from FedAvg, all other defense methods maintain relatively low ASR under BD attacks. However, for more sophisticated MR and LIE attacks, AFLGuard, FedCPA, and FedSSuper perform substantially worse than BRAFed. For example, under MR attacks, BRAFed reduces the average ASR across the three datasets to approximately 9.6%, representing a significant improvement over AFLGuard (72.3%), FedCPA (53.7%), and FedSSuper (46.7%). On the more complex COVIDx dataset, BRAFed attains an average ACC of around 81.3% across all six attack types, substantially exceeding the 69.5% average ACC typically achieved by the best state-of-the-art defense, FedSSuper.

Furthermore, we vary the proportion of malicious clients to 10%, 20%, 30%, 40%, and 45%, and evaluate the PR of the four defense

methods under all six attack types. The results on the CIFAR-100 dataset are illustrated in Fig. 4. As the attack ratio increases, the PR values of all methods decrease. Notably, AFLGuard, FedCPA, and FedSSuper experience the largest decline in PR under MR and LIE attacks, indicating that these methods cannot effectively adapt to such attacks. Across all attack types, BRAFed exhibits the smallest decline in PR, demonstrating that the proposed method is resilient to both data and model poisoning attacks, and consistently outperforms existing state-of-the-art defenses.

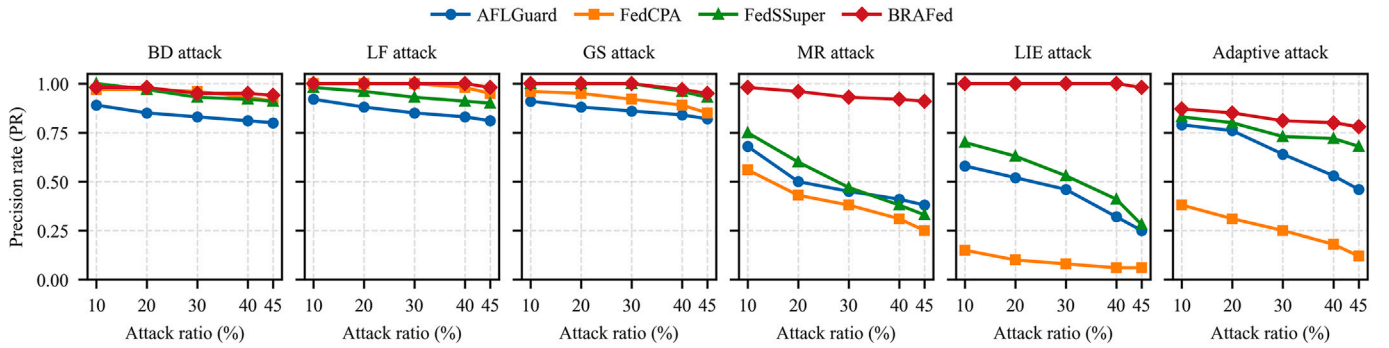
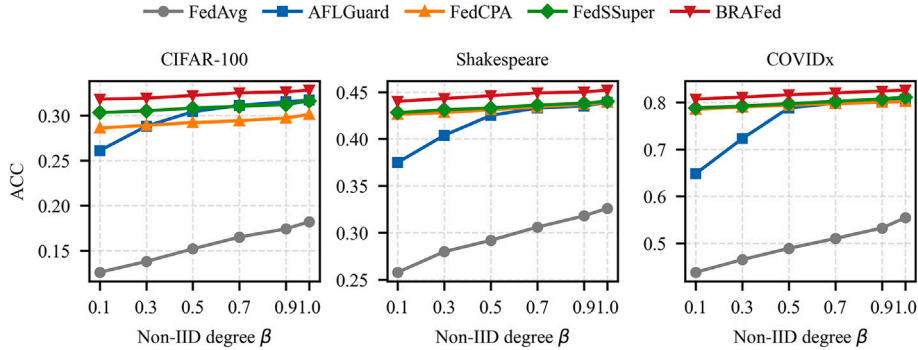
5.2.4. Performance on non-IID data

In this section, we adopt the default parameter setting with an attack ratio of 30% and a delay ratio of 40%. We compare BRAFed with the four benchmark methods in terms of ACC under varying degrees of data heterogeneity, with the Non-IID degree β ranging from 0.1 to 1. Since all methods exhibit consistent trends across different Non-IID degrees

Table 8

Results (ACC and ASR) of different detection methods for different attacks under three benchmark datasets.

Dataset	Method	BD attack		LF attack		GS attack		MR attack		LIE attack		Adaptive attack
		ACC	ASR	ACC	ACC	ACC	ASR	ACC	ASR	ACC	ASR	
CIFAR-100	FedAvg	0.287	1	0.236	0.126	0.167	0.98	0.153	0.96	0.161		
	AFLGuard	0.292	0.18	0.301	0.261	0.172	0.76	0.165	0.68	0.263		
	FedCPA	0.316	0.04	0.294	0.286	0.198	0.65	0.194	0.77	0.147		
	FedSSuper	0.32	0.06	0.306	0.303	0.223	0.52	0.236	0.62	0.294		
	BRAFed	0.327	0.06	0.315	0.318	0.303	0.08	0.314	0.04	0.313		
Shakespeare	FedAvg	0.418	0.92	0.376	0.258	0.232	0.88	0.238	0.98	0.254		
	AFLGuard	0.422	0.12	0.402	0.375	0.211	0.64	0.184	0.71	0.362		
	FedCPA	0.432	0.05	0.423	0.426	0.243	0.43	0.23	0.49	0.169		
	FedSSuper	0.436	0.04	0.43	0.428	0.298	0.39	0.274	0.55	0.417		
	BRAFed	0.448	0.01	0.445	0.44	0.437	0.1	0.446	0.07	0.43		
COVIDx	FedAvg	0.799	0.89	0.632	0.438	0.507	0.95	0.403	0.84	0.484		
	AFLGuard	0.767	0.12	0.795	0.648	0.386	0.77	0.372	0.64	0.582		
	FedCPA	0.812	0.06	0.802	0.785	0.434	0.53	0.421	0.44	0.205		
	FedSSuper	0.815	0.05	0.803	0.788	0.493	0.49	0.487	0.37	0.783		
	BRAFed	0.823	0.08	0.819	0.807	0.812	0.11	0.809	0.1	0.808		

**Fig. 4.** Comparison of defense performance on CIFAR-100 dataset with different attack ratios.**Fig. 5.** Model accuracy of five methods under varying Non-IID levels with Gaussian attacks.

under the six attack scenarios, we present only the results under GS attacks in Fig. 5.

It can be observed that as the Non-IID degree β increases, the ACC of all methods improves due to the reduction in data heterogeneity. FedCPA, FedSSuper, and BRAFed consistently maintain high ACC values with relatively small growth, as these methods incorporate mechanisms to handle heterogeneous data. In contrast, AFLGuard lacks such mechanisms and therefore exhibits a noticeable ACC gap compared to the other three defenses when $\beta \leq 0.5$, corresponding to a highly Non-IID setting. Across all Non-IID degrees, BRAFed achieves the highest ACC, owing to its integrated design that simultaneously addresses malicious gradient detection, delayed gradient correction, and data heterogeneity—capabilities not present in the other defense methods.

5.2.5. Performance on different delay ratios

In this section, we employ the default parameter settings, with the Non-IID degree $\beta = 0.1$ and the attack ratio set to 30%. We compare BRAFed with the four benchmark methods in terms of ACC under varying delay ratios δ from 0 to 0.6. Due to space constraints, we present the ACC results only for GS, MR, and Adaptive attacks on the Shakespeare dataset, as shown in Fig. 6.

An overarching trend is that ACC decreases for all methods as δ increases, demonstrating that delayed gradients indeed negatively impact model performance. However, across the three attack scenarios, AFLGuard and BRAFed exhibit a more moderate reduction in ACC compared to the other three methods, indicating superior resilience to delayed clients. This can be attributed to their dedicated mechanisms for

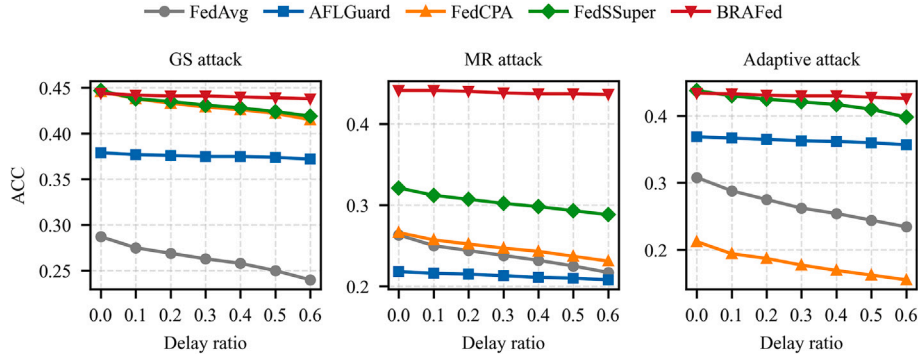


Fig. 6. Model accuracy of five methods under three attacks on Shakespeare dataset.

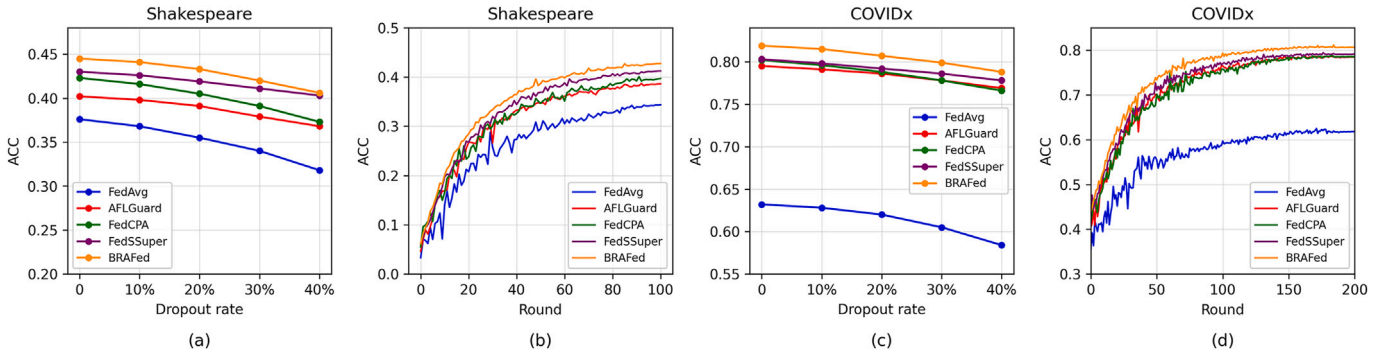


Fig. 7. Model accuracy of five methods under dynamic client participation.

handling delayed gradients: AFLGuard adopts an asynchronous update strategy to mitigate synchronization delays, whereas BRAFed incorporates a projection strategy to correct delayed updates. In contrast, the other three methods lack explicit mechanisms to address delayed updates.

For example, under MR attack, as the delay ratio increases from 0 to 0.6, ACC decreases by only 1% for AFLGuard and 0.5% for BRAFed, compared to 4.6% for FedAvg, 3.5% for FedCPA, and 3.3% for FedSSuper. The gap becomes even more pronounced under the more sophisticated Adaptive attack, where the accuracy reductions are 1.2% and 0.8% for AFLGuard and BRAFed, respectively, versus 7.4%, 5.7%, and 4% for FedAvg, FedCPA, and FedSSuper. This is because delayed gradients introduce directional bias, and adaptive attack gradients are optimized based on the current global model; the combination of these effects causes the aggregated gradients to deviate further from the optimal direction, leading to a larger accuracy drop under adaptive attacks compared to simpler attacks such as MR or LF attacks.

Overall, BRAFed maintains the most stable performance across different delay ratios, demonstrating its superior adaptability to both data heterogeneity and device heterogeneity under poisoning attacks.

5.2.6. Performance under client dropout and join

To further evaluate BRAFed under realistic dynamic client participation, we simulate selected-client dropout and client joining. Due to space limitations, here we only report representative results on the Shakespeare and COVIDx datasets under LF attack, while similar trends were also observed under other experimental settings. Unless otherwise stated, we set $\beta = 0.1$, $\sigma = 30\%$, $\delta = 40\%$.

We first evaluate selected-client dropout. In each round, the server samples a fixed proportion of clients, while each selected client fails to return its update with probability p_{drop} . We vary p_{drop} from 0 to 0.4. As shown in Fig. 7(a) and (c), the ACC of all methods decreases as the dropout rate increases, since fewer client updates are available for aggregation. FedAvg performs the worst because it directly averages the

Table 9

Computational overhead of different methods under LIE attack.

Method	CIFAR-100	Shakespeare	COVIDx
FedAvg	9.63s	159.67s	20.86s
AFLGuard	10.75s	181.45s	23.79s
FedCPA	14.13s	209.67s	28.89s
FedSSuper	11.34s	208.87s	25.96s
BRAFed	12.28s	200.8s	27.05s

returned updates without any defense or Non-IID correction. FedCPA also shows a relatively fast decline, as its critical-parameter-similarity-based detection requires enough client updates to form a stable reference. When dropout becomes severe, this reference becomes less reliable. AFLGuard and FedSSuper degrade more slowly because they rely on a trusted dataset and a public shadow dataset, respectively, which provide external references for malicious-update detection. BRAFed achieves the highest ACC under all dropout rates. Although its MD-based detection is also affected by the reduced number of returned gradients, the adaptive threshold, model-difference regularization, and projection-based correction help maintain stable performance.

We then evaluate client join dynamics with $p_{drop} = 20\%$. The total number of clients is fixed, but only 70% of clients are initially available. The remaining 30% gradually join the system during training and receive the latest global model after joining. We consider two join intervals: early join during $[0.1T, 0.3T]$ and late join during $[0.6T, 0.8T]$, where T refers to the number of global training rounds. As shown in Fig. 7(b) and (d), all methods show larger ACC fluctuations when clients join in the early stage, while the fluctuations are much smaller when clients join in the later stage. This is because newly joined clients mainly change the data distribution of the available client pool. In the early stage, the global model and gradient distribution are still unstable, so new clients have a visible influence on training stability. However, as the model converges, their influence becomes very small, and the newly joined clients hardly reduce the final convergence ACC.

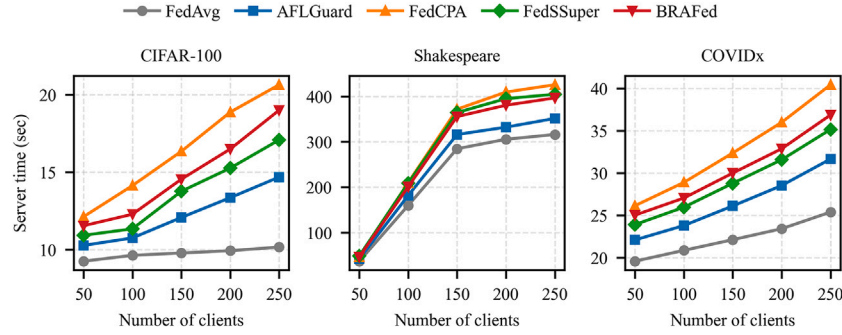


Fig. 8. Server per-Round computational overhead on three datasets under varying client numbers.

5.2.7. Computational overhead

In this section, we evaluate the computational overhead under the default parameter settings ($\beta = 0.1$, attack ratio = 30%, and 100 clients). Although servers typically possess greater computational resources, efficiently processing and aggregating high-dimensional model parameters from hundreds or thousands of clients simultaneously is critical, particularly in scenarios with frequent aggregation and updates.

Except for the baseline FedAvg, the other four defense methods primarily focus on server-side processing to detect and remove malicious clients. Therefore, in this evaluation, we only consider the computation time on the server side, excluding local training time and parameter transmission time. Table 9 reports the average server-side time per training round under LIE attack for the five methods across the three datasets. It can be observed that, compared with FedAvg without any defense mechanism, all four defense methods incur additional computational overhead. Specifically, BRAFed increases server computation time relative to FedAvg by 27.52% (CIFAR-100), 25.76% (Shakespeare), and 29.68% (COVIDx), which is within an acceptable range. Compared with the state-of-the-art method FedCPA, BRAFed achieves higher computational efficiency on the three datasets by 8.38% (CIFAR-100), 4.23% (Shakespeare), and 5.64% (COVIDx). Although BRAFed is on average 2.88% slower than FedSSuper, it consistently outperforms FedSSuper in both model accuracy and malicious client detection.

Furthermore, we evaluate the server-side time per training round for all five methods with varying client numbers: 50, 100, 150, 200, and 250. The results across the three datasets are shown in Fig. 8. It can be seen that the server time for all methods increases linearly with the number of clients. The growth rate of BRAFed is lower than that of FedCPA and comparable to FedSSuper, demonstrating that BRAFed is highly feasible and practical in resource-constrained and large-scale distributed environments.

6. Conclusion and future work

In this paper, we propose a Byzantine-robust asynchronous federated learning method on Non-IID Data, called BRAFed. Specifically, BRAFed limits the model drift induced by Non-IID data by integrating a model difference regularization term into the client-side local loss function. Furthermore, it detects malicious gradients by means of an adaptively adjusted MD threshold and further corrects any delayed gradients that conflict with the current update direction using a vector projection technique. This design effectively mitigates both poisoning attacks and the straggler problem in asynchronous FL. We provide a convergence analysis for BRAFed, revealing how delayed gradients, malicious gradients, and Non-IID data impact the convergence rate and model utility. We evaluate BRAFed against several state-of-the-art methods across three datasets under three attack scenarios. The results demonstrate that BRAFed effectively addresses both data heterogeneity and system heterogeneity, while outperforming existing approaches in terms of Byzantine robustness and model accuracy.

An interesting direction for future work is to explore the integration of generative models in FL. Generative models such as GANs or VAEs could generate synthetic data that mimic real data while preserving privacy [44]. This would allow FL systems to share privacy-preserving synthetic data, addressing privacy concerns more effectively. Another interesting direction for future work is to incorporate explicit network-level simulations, such as packet loss, latency variance, and bandwidth fluctuations, to further evaluate the robustness and practicality of BRAFed under more realistic communication environments.

CRedit authorship contribution statement

Chen Fang: Writing – original draft, Visualization, Validation, Software, Project administration, Funding acquisition, Conceptualization. **Fang Yang:** Validation, Project administration, Funding acquisition, Formal analysis. **Sensen Li:** Visualization, Validation, Supervision, Software, Data curation. **Kunyu Fan:** Validation, Software, Investigation, Data curation, Conceptualization. **Yuanbo Guo:** Supervision, Resources, Project administration, Funding acquisition. **Yanmi Wu:** Visualization, Supervision, Formal analysis, Data curation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work was supported by the National Cryptologic Science Fund of China (No. 2025NCSF02020) and the Key Science and Technology Program of Henan Province (No. 262102211088).

Data availability

Data will be made available upon request.

References

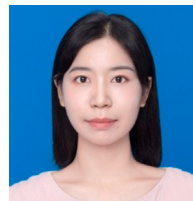
- [1] B. McMahan, E. Moore, D. Ramage, S. Hampson, B.A. Y. Arcas, Communication-efficient learning of deep networks from decentralized data, in: *Artificial Intelligence and Statistics*, PMLR, 2017, pp. 1273–1282.
- [2] C. Fang, Y. Guo, N. Wang, A. Ju, Highly efficient federated learning with strong privacy preservation in cloud computing, *Comput. Secur.* 96 (2020) 101889.
- [3] P. Mai, R. Yan, Y. Pang, Rflpa: a robust federated learning framework against poisoning attacks with secure aggregation, *Adv. Neural Inf. Process. Syst.* 37 (2024) 104329–104356.
- [4] Y. Miao, D. Kuang, X. Li, S. Xu, H. Li, K.-K.R. Choo, R.H. Deng, Privacy-preserving asynchronous federated learning under non-IID settings, *IEEE Trans. Inf. Forensics Secur.* 19 (2024) 5828–5841.
- [5] C. Xie, S. Koyejo, I. Gupta, Asynchronous federated optimization, *arXiv preprint arXiv:1903.03934*, 2019.
- [6] J. Cheng, P. Luo, N. Xiong, J. Wu, AAF: asynchronous-adaptive federated learning in edge-based wireless communication systems for countering communicable infectious diseases, *IEEE J. Sel. Areas Commun.* 40 (11) (2022) 3172–3190.

- [7] H. Zhu, Y. Zhou, H. Qian, Y. Shi, X. Chen, Y. Yang, Online client selection for asynchronous federated learning with fairness consideration, *IEEE Trans. Wirel. Commun.* 22 (4) (2022) 2493–2506.
- [8] Y. Miao, Z. Liu, X. Li, M. Li, H. Li, K.-K.R. Choo, R.H. Deng, Robust asynchronous federated learning with time-weighted and stale model aggregation, *IEEE Trans. Dependable Secure Comput.* 21 (4) (2023) 2361–2375.
- [9] J. Liu, J. Jia, T. Che, C. Huo, J. Ren, Y. Zhou, H. Dai, D. Dou, Fedasmu: efficient asynchronous federated learning with dynamic staleness-aware model update, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, 38, 2024, pp. 13900–13908.
- [10] S. Liu, L. Zhu, W. Zhang, Y. Miao, T. Leng, K.-K.R. Choo, Accuracy-improved privacy-preserving asynchronous federated learning in IoT, *IEEE Internet Things J.* 12 (9) (2025) 11733–11743.
- [11] Y. Yan, X. Tong, S. Wang, Clustered federated learning in heterogeneous environment, *IEEE Trans. Neural Netw. Learn. Syst.* 35 (9) (2023) 12796–12809.
- [12] C. Sun, T. Jiang, D. Pompili, Heterogeneous federated learning via generative model-aided knowledge distillation in the edge, *IEEE Internet Things J.* 12 (5) (2025) 5589–5599.
- [13] Y. Gong, Y. Li, N.M. Freris, FedADMM: a robust federated deep learning framework with adaptivity to system heterogeneity, in: *2022 IEEE 38th International Conference on Data Engineering (ICDE)*, IEEE, 2022, pp. 2575–2587.
- [14] Z. Zhou, Y. Li, X. Ren, S. Yang, Towards efficient and stable K-asynchronous federated learning with unbounded stale gradients on non-IID data, *IEEE Trans. Parallel Distrib. Syst.* 33 (12) (2022) 3291–3305.
- [15] Q. Dong, Y. Gao, C. Zhou, S. Yang, B. Kuang, A. Fu, SmartFL: simple majority rule based byzantine-robust federated learning, *Inf. Fusion* (2025) 103555.
- [16] A. Song, T. Zhang, K. Cheng, Y. Cao, X. Zhu, Y. Shen, Byzantine-robust federated learning framework via a server-client defense mechanisms, *IEEE Internet Things J.* 12 (14) (2025) 29073–29088.
- [17] Y.-R. Yang, W.-J. Li, Buffered asynchronous SGD for byzantine learning, *J. Mach. Learn. Res.* 24 (204) (2023) 1–62.
- [18] M. Fang, J. Liu, N.Z. Gong, E.S. Bentley, Aflguard: byzantine-robust asynchronous federated learning, in: *Proceedings of the 38th Annual Computer Security Applications Conference*, 2022, pp. 632–646.
- [19] Y. Bai, Y. Wang, X. Xu, Y. Yang, H. Batool, Z. Iqbal, J. Xu, AsyncDefender: dynamic trust adaptation and collaborative defense for byzantine-robust asynchronous federated learning, *Comput. Netw.* (2025) 111430.
- [20] H.-G. Joo, S. Hong, D.-J. Shin, FedLSC: improving communication efficiency and robustness in federated learning with stragglers and adversaries, *IEEE Trans. Neural Netw. Learn. Syst.* 36 (11) (2025) 19805–19819.
- [21] H. Ma, K. Yang, Y. Jiao, Cellular traffic prediction via byzantine-robust asynchronous federated learning, *IEEE Trans. Netw. Sci. Eng.* 12 (4) (2025) 2402–2414.
- [22] P. Blanchard, E.M. El Mhamdi, R. Guerraoui, J. Stainer, Machine learning with adversaries: byzantine tolerant gradient descent, *Adv. Neural Inf. Process. Syst.* 30 (2017).
- [23] D. Yin, Y. Chen, R. Kannan, P. Bartlett, Byzantine-robust distributed learning: towards optimal statistical rates, in: *International Conference on Machine Learning*, PMLR, 2018, pp. 5650–5659.
- [24] L. Shen, Z. Ke, J. Shi, X. Zhang, Y. Sun, J. Zhao, X. Wang, X. Zhao, SPEFL: efficient security and privacy-enhanced federated learning against poisoning attacks, *IEEE Internet Things J.* 11 (8) (2023) 13437–13451.
- [25] P. Sun, X. Liu, Z. Wang, B. Liu, Byzantine-robust decentralized federated learning via dual-domain clustering and trust bootstrapping, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 24756–24765.
- [26] H. Kasyap, S. Tripathy, Sine: similarity is not enough for mitigating local model poisoning attacks in federated learning, *IEEE Trans. Dependable Secure Comput.* 21 (5) (2024) 4481–4494.
- [27] Z. Gong, L. Shen, Y. Zhang, L.Y. Zhang, J. Wang, G. Bai, Y. Xiang, Agramplifier: defending federated learning against poisoning attacks through local update amplification, *IEEE Trans. Inf. Forensics Secur.* 19 (2023) 1241–1250.
- [28] V. Shejwalkar, A. Houmansadr, Manipulating the byzantine: optimizing model poisoning attacks and defenses for federated learning, in: *NDSS*, 2021.
- [29] Y. Jiang, W. Zhang, Y. Chen, Data quality detection mechanism against label flipping attacks in federated learning, *IEEE Trans. Inf. Forensics Secur.* 18 (2023) 1625–1637.
- [30] S. Han, S. Park, F. Wu, S. Kim, B. Zhu, X. Xie, M. Cha, Towards attack-tolerant federated learning via critical parameter analysis, in: *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023, pp. 4999–5008.
- [31] S. Zuo, X. Yan, R. Fan, H. Hu, H. Shan, T.Q.S. Quek, P. Zhao, Federated learning resilient to byzantine attacks and data heterogeneity, *IEEE Trans. Mob. Comput.* 24 (10) (2025) 10729–10742.
- [32] G. Chen, W. Wang, Y. Wu, C. Li, G. Xu, S. Ji, T. Li, M. Shen, Y. Han, RobustPFL: robust personalized federated learning, *IEEE Trans. Dependable Secure Comput.* 22 (4) (2025) 3224–3239.
- [33] Y. Qiao, C. Zhang, A. Adhikary, C.S. Hong, Logit calibration and feature contrast for robust federated learning on non-iid data, *IEEE Trans. Netw. Sci. Eng.* 12 (2) (2024) 636–652.
- [34] P. Zhao, J. Jiang, G. Zhang, FedSuper: a secure and private federated learning under trusted supervision, *IEEE Trans. Netw. Sci. Eng.* 12 (6) (2025) 4544–4559.
- [35] T. Li, A.K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, V. Smith, Federated optimization in heterogeneous networks, *Proc. Mach. Learn. Syst.* 2 (2020) 429–450.
- [36] Y. Hu, J. Zheng, S. Jiang, S. Yang, J. Zou, R. Wang, A mahalanobis distance-based approach for dynamic multiobjective optimization with stochastic changes, *IEEE Trans. Evol. Comput.* 28 (1) (2023) 238–251.
- [37] G. Yan, H. Wang, X. Yuan, J. Li, FedRoLA: robust federated learning against model poisoning via layer-based aggregation, in: *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2024, pp. 3667–3678.
- [38] L. Wang, Z.Q. Lin, A. Wong, COVID-Net: a tailored deep convolutional neural network design for detection of COVID-19 cases from chest x-ray images, *Sci. Rep.* 10 (1) (2020) 19549.
- [39] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 770–778.
- [40] T. Gu, B. Dolan-Gavitt, S. BadNets, Identifying vulnerabilities in the machine learning model supply chain, in: *Proceedings of the Neural Information Processing Symposium Workshop Mach. Learning Security (MLSec)*, 2017, pp. 1–5.
- [41] E. Bagdasaryan, A. Veit, Y. Hua, D. Estrin, V. Shmatikov, How to backdoor federated learning, in: *International Conference on Artificial Intelligence and Statistics*, PMLR, 2020, pp. 2938–2948.
- [42] G. Baruch, M. Baruch, Y. Goldberg, A little is enough: circumventing defenses for distributed learning, *Adv. Neural Inf. Process. Syst.* 32 (2019).
- [43] M. Fang, X. Cao, J. Jia, N. Gong, Local model poisoning attacks to (byzantine-robust) federated learning, in: *29th USENIX Security Symposium (USENIX Security 20)*, 2020, pp. 1605–1622.
- [44] T. Wang, Y. Zhang, S. Qi, R. Zhao, Z. Xia, J. Weng, Security and privacy on generative data in AIGC: a survey, *ACM Comput. Surv.* 57 (4) (2024) 1–34.

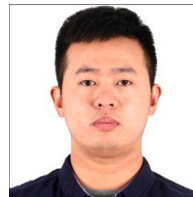
Author biography



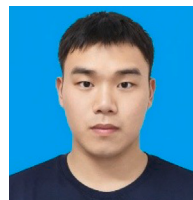
Chen Fang received the Ph.D. degree in software engineering from Information Engineering University, Zhengzhou, China. He is currently a lecturer at Information Engineering University. He has over 20 publications, including patents, journal articles, and conference papers. His research interests focus on addressing security and privacy issues in federated learning.



Fang Yang received the Ph.D. degree in applied mathematics from Huazhong University of Science and Technology, Wuhan, China. She is currently an associate professor at Information Engineering University. Her research interests focus on homomorphic encryption and convergence analysis.



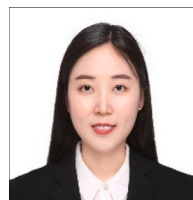
Sensen Li received his Ph.D. degree in computer science from Information Engineering University, Zhengzhou, China. He is currently an associate professor at Information Engineering University. His research interests focus on data security.



Kunyuan Fan is currently pursuing his bachelor's degree at the Information Engineering University. His research interests focus on addressing security and privacy issues in federated learning.



Yuanbo Guo received the Ph.D. degree in Computer Science from Xidian University, Xi'an, China. He is currently a Professor at Hainan University. His research interests focus on networking and information security.



Yanmi Wu received the Ph.D. degree in applied mathematics from Zhengzhou University, Zhengzhou, China. She is a lecturer at Information Engineering University. Her research interests focus on homomorphic encryption and convergence analysis.